Univerzitet u Sarajevu

Elektrotehnički fakultet

Verifikacija i validacija softvera - Projektni zadatak – II dio

Projekat radili:

Rubina Rekić

Irma Lemeš

Irma Topčagić

Nedim Omanović

Sadržaj

1 Metoda FiltrirajPretraziFilmove() - Irma Topčagić **3**
1.1 McCabe metrika . . . 4
1.1.1 Proračun McCabe metrike . . . 4
1.1.2 Kontrolni graf . . . 5
1.1.3 Rezultati i diskusija . . . 6
1.2 White box testiranje . . . 7
1.2.1 Obuhvat iskaza/linija (Statement/line coverage) . . . 7
1.2.2 Obuhvat odluka/grana . . . 7
1.2.3 Obuhvat uslova . . . 8
1.2.4 Modifikovani uslov/odluka obuhvat (MCDC) . . . 9
1.2.5 Obuhvat petlji (Loop Coverage) . . . 9
1.2.6 Obuhvat toka podataka (Data flow coverage) . . . 9
1.2.7 Obuhvat puteva (Path coverage) . . . 10
1.3 Sinteza rezultata White-box testiranja . . . 11
1.4 Code Tuning . . . 11
1.4.1 Korištene tehnike . . . 11
1.4.2 Tuning petlji . . . 12
1.4.3 Tuning logičkih iskaza – Early Exit . . . 13
1.4.4 Tuning izraza - Optimizacija LINQ upita . . . 13
1.4.5 Kodovi za Code Tuning . . . 14
1.4.6 Memorija . . . 17
1.4.7 Utjecaj na McCabe metriku i indeks održavanja . . . 18
1.5 Refaktoring . . . 19
1.5.1 Primijenjene stavke iz check liste . . . 19
1.5.2 Refaktorisani kod . . . 21
1.5.3 Utjecaj refaktorisanja na metrike . . . 22

2 Metoda DodajFilm() - Rubina Rekić **23**
2.1 Opis metode . . . 23
2.2 McCabe metrika . . . 24
2.2.1 Proračun McCabe metrike . . . 24
2.2.2 Kontrolni graf . . . 25
2.2.3 Poređenje rezultata . . . 26
2.3 White-box testiranje . . . 27
2.3.1 Obuhvat iskaza/linija . . . 27
2.3.2 Obuhvat odluka/grana . . . 28
2.3.3 Obuhvat uslova . . . 29
2.3.4 Obuhvat petlji . . . 30
2.3.5 Obuhvat toka podataka . . . 31
2.3.6 Obuhvat puteva . . . 31
2.4 Code tuning . . . 32
2.4.1 Problem pri tuningu i rješenje . . . 32
2.4.2 Izmještanje koda izvan petlje (Moving Invariants) . . . 33
2.4.3 Logičko pojednostavljivanje . . . 36
2.4.4 Izbor boljeg algoritma ili struktura podataka . . . 37
2.5 Refaktoring . . . 39
2.5.1 Checklist i očekivani rezultati refaktoringa . . . 39
2.5.2 Replace Magic Number with Symbolic Constant . . . 39
2.5.3 Rename Variable (Preimenovanje) . . . 40
2.5.4 Remove Control Flag . . . 40
2.5.5 Extract Method . . . 40
2.5.6 Pregled postignutih rezultata . . . 42

3 Metoda obrisiFilm() - Nedim Omanović **43**
3.1 Opis metode . . . 43
3.2 McCabe metrika . . . 45
3.2.1 Proračun McCabe metrike . . . 45
3.2.2 Kontrolni graf . . . 46
3.2.3 Poređenje rezultata . . . 47
3.3 White-box testiranje . . . 48
3.3.1 Obuhvat iskaza/linija . . . 48
3.3.2 Obuhvat odluka/grana . . . 49
3.3.3 Obuhvat uslova . . . 50
3.3.4 Obuhvat petlji . . . 51
3.3.5 Obuhvat toka podataka . . . 52
3.3.6 Obuhvat puteva . . . 52
3.4 Code tuning . . . 53
3.4.1 Problem pri tuningu i rješenje . . . 53
3.4.2 Izmještanje koda izvan petlje (Moving Invariants) . . . 54
3.4.3 Logičko pojednostavljivanje . . . 57
3.4.4 Izbor boljeg algoritma ili struktura podataka . . . 58
3.5 Refaktoring . . . 60
3.5.1 Checklist i očekivani rezultati refaktoringa . . . 60
3.5.2 Replace Magic String with Symbolic Constant . . . 60
3.5.3 Rename Variable (Preimenovanje) . . . 60
3.5.4 Remove Control Flag . . . 61
3.5.5 Extract Method . . . 61
3.5.6 Pregled postignutih rezultata . . . 63

4 Metoda OcijeniFilmGledaoca() - Irma Lemeš **65**
4.1 Opis metode . . . 65
4.2 McCabe metrika . . . 66
4.2.1 Proračun McCabe metrike . . . 66
4.2.2 Kontrolni graf . . . 66
4.2.3 Poređenje rezultata . . . 68
4.3 White box testiranje . . . 68
4.3.1 Obuhvat iskaza/linija (Statement/line coverage) . . . 68
4.3.2 Obuhvat odluka/grana (Decision/Branch Coverage) . . . 70
4.3.3 Obuhvat uslova (Condition Coverage) . . . 71
4.3.4 Obuhvat petlji (Loop Coverage) . . . 72
4.3.5 Obuhvat toka podataka (Data Flow Coverage) . . . 73
4.3.6 Obuhvat puteva (Path Coverage) . . . 74
4.4 Code Tuning . . . 75
4.4.1 Test-verzija metode . . . 75
4.4.2 Korištene tehnike . . . 75
4.4.3 Tuning pristupa kolekcijama . . . 76
4.4.4 Tuning string izraza . . . 77
4.4.5 Tuning algoritma pretrage . . . 78
4.4.6 Uticaj na metrike koda . . . 79
4.5 Refaktoring . . . 80
4.5.1 Checklist i očekivani rezultati refaktoringa . . . 80
4.5.2 Replace Magic Number with Symbolic Constant . . . 80
4.5.3 Replace Conditional with Guard Clauses . . . 81
4.5.4 Extract Method . . . 81
4.5.5 Pregled postignutih rezultata . . . 82

1 Metoda FiltrirajPretraziFilmove() - Irma Topčagić

Analizira se metoda `FiltrirajPretraziFilmove()` iz klase `FilmService`.Metoda omogućava pretragu i filtriranje filmova u sistemu na osnovu izbora(naziv, kategorija ili minimalna ocjena), te vraća listu nađenih filmova, ili praznu ukoliko nema rezultata ili je pogrešan unos.

Kod metode:

```
    public virtual List<Film> FiltrirajPretraziFilmove( string izbor,string? naziv=null,string?
    ↪  kategorija=null,double minimalnaOcjena=0)
{

    List<Film> sviFilmovi = new List<Film>();
    foreach (var film in filmRepo.GetAll())
    {
        sviFilmovi.Add(film);
    }

    if (sviFilmovi.Count == 0)
    {
        return new List<Film>();
    }

    List<Film> rezultati = new List<Film>();

    if (izbor == "1")
    {
        rezultati = sviFilmovi
            .Where(f => f.getNazivFilma().Contains(naziv ?? "Nepostojeci film"))
            .ToList();
    }
    else if (izbor == "2")
    {
        rezultati = sviFilmovi
            .Where(f => f.getKategorija() == kategorija)
            .ToList();
    }
    else if (izbor == "3")
    {
        rezultati = sviFilmovi
            .Where(f => f.getOcjena() >= minimalnaOcjena)
            .ToList();
    }
    else if (izbor == "0")
    {
        return new List<Film>();
    }
    else
    {
        return new List<Film>();
    }


    if (rezultati.Any())
    {
        if (rezultati.Count > 10)
```

```
        {
            return rezultati.Take(10).ToList();
        }

        else { return rezultati; }
    }


    return new List<Film>();
}
```

1.1 McCabe metrika

1.1.1 Proračun McCabe metrike

Formula koja će biti primjenjena za računanje McCabe metrike je sljedeća:

$$V(G) = e - n + 2p$$

Značenje parametara je sljedeće:

1. **e** je broj ivica(tokova) u grafu programskog toka,
2. **n** je broj čvorova u grafu programskog toka,
3. **p** je broj formiranih neovisnih dijelova grafa.

Konkretno, u kontekstu metode koja se analizira, imamo da je:

1. **p=1** jer je u pitanju analiza jedne metode,
2. **n=15** čvorova u kontrolnom grafu,
3. **e=23** usmjerenih grana u kontrolnom grafu

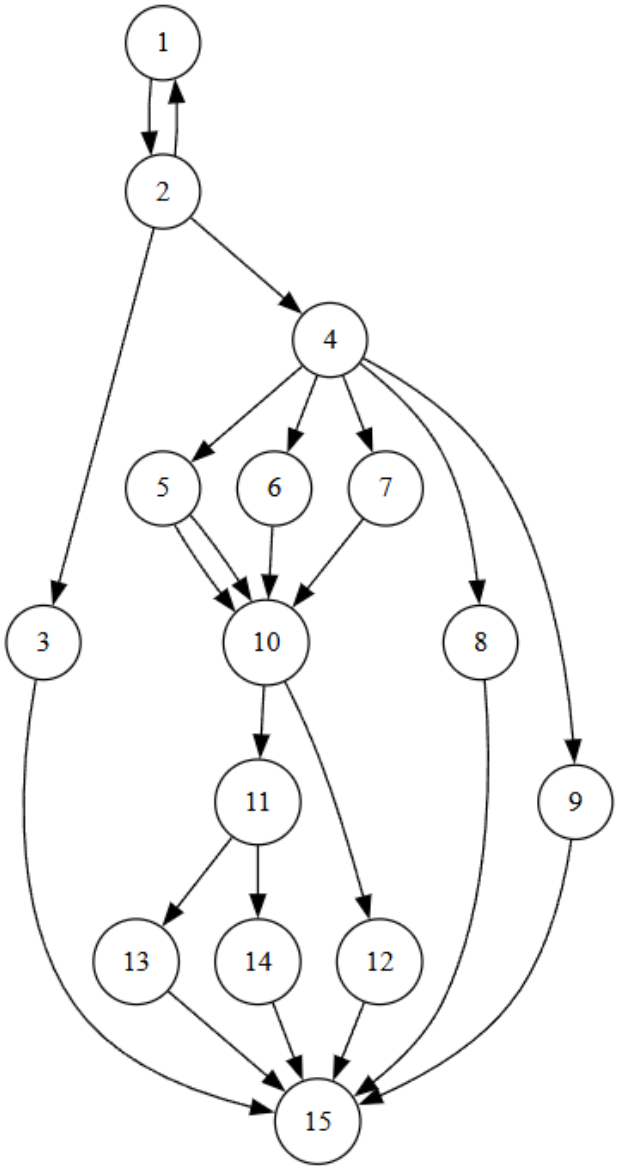
Uvrštavanjem u formulu dobijamo:

$$V(G) = 23 - 15 + 2 * 1$$

$$V(G) = 10 \tag{1}$$

1.1.2 Kontrolni graf

U prilogu se nalazi kontrolni graf.



Slika 1: Kontrolni graf

1.1.3 Rezultati i diskusija

Rezultati ručnog proračuna upoređeni su s proračunom iz Code Metrics alata - dobija se isti rezultat.

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmMate (Debug)	83	233	2	37	1.170	420
Program	54	35	1	20	203	82
FilmMate.Services	73	150	1	24	766	277
FilmService	60	129	1	10	644	238
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	46	13		4	70	30
azurirajFilm() : void	56	7		4	39	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4
OcijeniFilmGledaoca() : void	51	10		4	57	22
FiltrirajPretraziFilmove(string, string?, string?, dou	48	10		6	57	26
filtrirajPoNazivu(List<Film>) : List<Film>	66	4		6	16	7
filtrirajPoKategoriji(List<Film>) : List<Film>	66	4		6	16	7
filtrirajPoMinimalnojOcjeni(List<Film>) : List<Film>	63	4		6	22	9

Slika 2: Rezultati iz Code Metrics

Kvalitet koda: Vrijednost $V(G) = 10$ spada u opseg 10–20, što se prema Jonesovim pronalascima smatra alarmom za pojačanu kontrolu i podjelu modula. Ova metoda se ne smatra jednostavnom i razumljivom, već zahtijeva pojačanu pažnju tokom testiranja i održavanja.

Poteškoće u testiranju: Visoka složenost otežava testiranje, te povećava broj potrebnih testova kako bi se obuhvatili svi slučajevi.

Indeks održivosti (MI = 48): Metoda se uklapa u kategoriju "teška za održavanje" (ispod granice od 65), što znači da je podložna greškama i teška za izmjenu.
Visoka složenost metode (57 linija koda) proizlazi iz logičkih grana, ponavljajuće `if-else if` strukture i filtera s lambda izrazima. Ciklomatska složenost metode je povećana zbog kombinacije grananja za izbor filtriranja (naziv, kategorija, minimalna ocjena) i dodatne logike za ograničenje rezultata na maksimalno 10 elemenata. Metoda je funkcionalno jasna, ali zbog broja linija i grananja, promjene u logici mogu biti podložne greškama, a održavanje zahtijeva pažljivo praćenje toka programa.

1.2 White box testiranje

1.2.1 Obuhvat iskaza/linija (Statement/line coverage)

Cilj obuhvata iskaza (statement/line coverage) je osigurati da se svaka izvršna linija koda u metodi `FiltrirajPretraziFilmove()` izvrši barem jednom. Za postizanje 100% obuhvata linija, potreban je minimalan broj od 6 testnih slučajeva. U nastavku je definisano 6 testnih slučajeva koji garantuju prolazak kroz sve programske linije.

Tablica 1: Test slučajevi T1–T6 (Obuhvat iskaza)

Test ID	Ulaz	Očekivan izlaz	Opis testa
TC1	Bilo kakav unos filmRepo: Prazna lista	Vraća praznu listu	Provjera prvog `if` uslova kada je repozitorij prazan.
TC2	Unos: `"0"`	Povratak na glavni meni (metoda završava)	Korisnik bira opciju za izlaz. Pokriva `else if (izbor == "0")`.
TC3	Unos: `"1", naziv: "Matrix"` filmRepo sadrži > 10 filmova	Lista sa 10 filmova	Pretraga po nazivu vraća više od 10 rezultata. Pokriva granu `Count > 10`.
TC4	Unos: `"2", kategorija:"Akcija"` Repo sadrži 5 filmova	Lista od 5 filmova	Filtriranje po kategoriji vraća manje od 10 rezultata. Pokriva `else` granu limita.
TC5	Unos: `"3", minimalnaOcjena:5.2` (filmRepo ne sadrži filmove s tom ocjenom)	Vraća praznu listu	Filtriranje po ocjeni ne vraća ništa. Pokriva zadnji return kada nije ispunjen if `Any()`.
TC6	Unos: `"abc"` (Pogrešan unos)	Prazna lista	Pokriva završni `else` blok.

Analiza pokrivenosti: Ovim skupom testova izvršene su sve linije koda:

- **TC1** pokriva rani izlaz.
- **TC2** pokriva izlaz iz menija.
- **TC3, TC4, TC5** pokrivaju sve tri validne opcije pretrage (”1”, ”2”, ”3”) i sve tri moguće situacije sa rezultatima (puno, malo, ništa).
- **TC6** pokriva pogrešan izbor (niti jedna od opcija ”0”,”1”,”2”,”3”).

1.2.2 Obuhvat odluka/grana

Cilj je dizajnirati minimalan broj testnih slučajeva koji osiguravaju da se svi mogući ishodi odluka (TRUE i FALSE) u metodi `FiltrirajPretraziFilmove()` izvrše barem jednom.

Metoda ima sljedeće ključne odluke:

- **D1:** Provjera je li lista prazna (`if (Count == 0)`).
- **D2:** Glavna odluka (odabir opcije `'1'`, `'2'`, `'3'`, `'0'` ili `pogrešan odabir`).
- **D3:** Provjera rezultata (`if (rezultati.Any())`).
- **D4:** Provjera limita (`if (rezultati.Count > 10)`).
- **D5:** Implicitna provjera null vrijednosti parametra `naziv` (upotreba operatora `??`)

Da bismo postigli 100% obuhvat odluka, potreban je skup od 7 testnih slučajeva.

Tablica 2: Test slučajevi T1–T7 (Obuhvat odluka/grana)

Test ID	**Ulaz**	**Očekivan izlaz**	**Opis testa**
TC1	`Bilo` kakav unos filmRepo: Prazna lista	Vraća praznu listu	Provjera prvog `if` uslova kada je repozitorij prazan.
TC2	Unos: `"0"`	Povratak na glavni meni (metoda završava)	Korisnik bira opciju za izlaz. Pokriva `else if (izbor == "0")`.
TC3	Unos: `"1", naziv: "Matrix"` filmRepo sadrži > 10 filmova	Lista sa 10 filmova	Pretraga po nazivu vraća više od 10 rezultata. Pokriva granu `Count > 10`.
TC4	Unos: `"2", kategorija:"Akcija"` Repo sadrži 5 filmova	Lista od 5 filmova	Filtriranje po kategoriji vraća manje od 10 rezultata. Pokriva `else` granu limita.
TC5	Unos: `"3", minimalnaOcjena:5.2` (filmRepo ne sadrži filmove s tom ocjenom)	Vraća praznu listu	Filtriranje po ocjeni ne vraća ništa. Pokriva zadnji return kada nije ispunjen if `Any()`.
TC6	Unos: `"abc"` (Pogrešan unos)	Prazna lista	Pokriva završni `else` blok.
TC7	Unos: `"1", naziv: null` (Repo sadrži filmove)	Prazna lista	Testira implicitnu granu operatora `??`. Null naziv se mijenja u nepostojeći string, rezultirajući praznom listom.

Kako bi se sistematično prikazao obuhvat odluka, identificirane su sve tačke grananja u metodi i označene oznakama od **D1** do **D5**, kao i prikazano da li iznose true ili false:

Tablica 3: Detaljni prikaz testova za obuhvat odluka (Decision/Branch Coverage)

ID	**Scenarij**	**Ulazni podaci**	**Pokrivene grane (ishodi)**
TC1	Nema filmova u bazi	`filmRepo` je prazan	**D1: True** (filmRepo prazan)
TC2	Izlaz (return)	Unos: `"0"`	D1: False, **D2: Case "0"**
TC3	Lista rezultata > 10	Unos: `"1"`, naziv: "Matrix"	D1: False, **D2: Case "1"**, **D5: True**, **D3: True**, **D4: True**
TC4	Malo rezultata (≤ 10)	Unos: `"2"`	D1: False, **D2: Case "2"**, **D3: True**, **D4: False**
TC5	Nema rezultata	Unos: `"3"`, Rezultat: 0	D1: False, **D2: Case "3"**, **D3: False**
TC6	Pogrešan unos	Unos: `"abc"`	D1: False, **D2: Default** (Else)
TC7	Null naziv	Unos: `"1"`, naziv: null	D1: False, **D2: Case "1"**, **D5: False** (Null check), **D3: False**

1.2.3 Obuhvat uslova

Na osnovu analize izvornog koda metode `FiltrirajPretraziFilmove()`, utvrđeno je sljedeće:

1. Svi odlučujući iskazi (`lanci if-else naredbi`) sastoje se od jednostavnih uslova (npr., `Count == 0`, `Count > 10`..).
2. U metodi ne postoje složeni logički izrazi koji koriste operatore kao što su logičko AND (&&) ili logičko OR (||).

Zbog jednostavnosti logičkih iskaza:

- Obuhvat odluka (Decision Coverage) zahtijeva testiranje ishoda cijelog izraza (TRUE i FALSE).

- Obuhvat uslova (Condition Coverage) zahtijeva testiranje svakog pojedinačnog uslova na TRUE i FALSE.

Budući da je svaki logički iskaz u metodi sastavljen od samo jednog uslova, postizanje **obuhvata odluka** automatski podrazumijeva i postizanje **obuhvata uslova**.

1.2.4 Modifikovani uslov/odluka obuhvat (MCDC)

Prema definiciji MCDC strategije, cilj je testirati utjecaj svakog pojedinačnog uslova unutar složene odluke na njen konačni ishod. Međutim, kada su odluke jednostavne (sastoje se od samo jednog uslova), MCDC kriterij je trivijalno zadovoljen i ekvivalentan obuhvatu odluka, jer svaka odluka sadrži tačno jedan uslov.

1.2.5 Obuhvat petlji (Loop Coverage)

Analizom koda identificirana je jednostavna petlja (`foreach`) koja iterira kroz listu filmova dobavljenu iz repozitorija. Za testiranje ove petlje primijenjena je strategija za jednostavne petlje, koja nalaže testiranje sljedećih scenarija: preskakanje petlje (0 prolaza), jedan prolaz, dva prolaza i m prolaza (tipičan slučaj).

S obzirom na to da broj iteracija ovisi o broju elemenata u repozitoriju, testni slučajevi su dizajnirani manipulacijom stanja repozitorija.

Tablica 4: Test slučajevi za obuhvat petlji (Loop Coverage)

Test ID	Ulaz (Stanje repozitorija)	Očekivani izlaz	Opis testa (Strategija)
TC_L1	`filmRepo` vraća praznu listu (Broj filmova = 0)	`sviFilmovi` ostaje prazna lista	**0 prolaza:** Tijelo petlje se u potpunosti preskače.
TC_L2	`filmRepo` vraća listu sa **1 filmom**	`sviFilmovi` sadrži tačno taj 1 film	**1 prolaz:** Tijelo petlje se izvršava jednom.
TC_L3	`filmRepo` vraća listu sa **2 filma**	`sviFilmovi` sadrži tačno ta 2 filma	**2 prolaza:** Tijelo petlje se izvršava dva puta. Provjera inicijalizacije za višestruke prolaze.
TC_L4	`filmRepo` vraća listu sa **5 filmova**	`sviFilmovi` sadrži svih 5 filmova	m **prolaza:** Petlja se izvršava slučajan broj puta ($m < n$). Predstavlja tipičan slučaj korištenja.

Napomena: S obzirom da `foreach` petlja nema eksplicitno definiran gornji limit (n) u samoj strukturi koda (limitirana je samo veličinom kolekcije), testiranje limita $n, n-1, n+1$ nije primjenjivo na isti način kao kod `for` petlji sa fiksnim brojačem. Scenarij **TC_L4** pokriva opći slučaj višestrukih iteracija.

1.2.6 Obuhvat toka podataka (Data flow coverage)

Obuhvat toka podataka (data flow coverage) fokusira se na praćenje putanja od mjesta definicije varijable do njenih stvarnih upotreba (DEF–USE putanje). U nastavku je data DEF–USE analiza ključnih varijabli metode `FiltrirajPretraziFilmove()`.

DEF–USE analiza varijabli:

- `sviFilmovi`
  - **DEF:** Inicijalizacija prazne liste i popunjavanje unutar `foreach` petlje.
  - **USE:** Provjera uslova `sviFilmovi.Count == 0` i korištenje u `Where()` izrazima.
- `rezultati`
  - **DEF:** Inicijalizacija prazne liste i redefinicija nakon filtriranja.
  - **USE:** Provjere `rezultati.Any()`, `rezultati.Count > 10` i ograničavanje rezultata pomoću `Take(10)`.
- `izbor`
  - **DEF:** Ulazni parametar metode.
  - **USE:** Grananje unutar `if--else if` strukture.

- `naziv`
  - **DEF:** Ulazni nullable parametar.
  - **USE:** Upotreba u izrazu `Contains(naziv ?? "...")`.
- `kategorija`
  - **DEF:** Ulazni nullable parametar.
  - **USE:** Poređenje vrijednosti u izrazu `f.getKategorija() == kategorija`.
- `minimalnaOcjena`
  - **DEF:** Ulazni numerički parametar.
  - **USE:** Poređenje u izrazu `f.getOcjena() >= minimalnaOcjena`.

Na osnovu prethodne analize definisano je 7 testnih slučajeva koji osiguravaju pokrivanje svih značajnih DEF–USE putanja.

Tablica 5: Testni slučajevi za obuhvat toka podataka (Data Flow Coverage)

Test ID	**Ulazni parametri**	**Očekivani izlaz**	**Pokrivene DEF–USE putanje**
TC1	`izbor="1"` `filmRepo = prazna lista`	Prazna lista	DEF `sviFilmovi` → USE `sviFilmovi.Count == 0` (rani izlaz)
TC2	`izbor="1"`, `naziv="Matrix"`	Lista od 10 filmova	DEF `rezultati` → USE `rezultati.Any()` (TRUE) DEF `rezultati.Count` → USE `Count > 10` (TRUE)
TC3	`izbor="2"`, `kategorija="Drama"`	Lista sa <10 filmova	DEF `rezultati` → USE `rezultati.Any()` (TRUE) DEF `rezultati.Count` → USE `Count > 10` (FALSE)
TC4	`izbor="3"`, `minimalnaOcjena=9.5`	Prazna lista	DEF `rezultati` → USE `rezultati.Any()` (FALSE)
TC5	`izbor="0"`	Prazna lista	DEF `izbor` → USE u odluci za izlaz iz metode
TC6	`izbor="abc"`	Prazna lista	DEF `izbor` → USE u završnom `else` bloku
TC7	`izbor="1", naziv=null`	Prazna lista	DEF `naziv` → USE u izrazu `naziv ?? ""` (NULL grana) DEF `rezultati` → USE `rezultati.Any()` (FALSE)

1.2.7 Obuhvat puteva (Path coverage)

Path coverage podrazumijeva pokrivanje svih izvedivih logičkih puteva kroz grananje.

- **P1:** `filmRepo` prazan → Prekid izvršavanja (**TC1**)
- **P2:** `izbor == "0"` → Povratak iz metode (**TC2**)
- **P3:** `izbor == "1"` (Ime) ∧ `Count > 10` → Lista 10 filmova (**TC3**)
- **P4:** `izbor == "2"` (Kategorija) ∧ `Count` $\leq$ `10` → Lista svih filmova (manje od 10) (**TC4**)
- **P5:** `izbor == "3"` (Ocjena) ∧ `Count == 0` → Prazna lista (**TC5**)
- **P6:** `izbor` nevalidan (else grana) → Prazna lista (**TC6**)
- **P7:** `izbor == "1"`, `naziv` null → Prazna lista (**TC7**)

Tablica 6: Testni slučajevi za obuhvat puteva (Path Coverage)

Test ID	Ulazni parametri	Očekivani izlaz	Opis testa	Putanja
TC1	`izbor="1", filmRepo = prazna lista`	Prazna lista	Lista prazna, rani izlaz iz metode	$1 \to 2 \to 3 \to 15$
TC2	`izbor="0", filmRepo = ne-prazna lista`	Prazna lista	Povratak iz metode zbog izbora "0"	$1 \to 2 \to 4 \to 8 \to 15$
TC3	`izbor="1", naziv="Matrix", filmRepo` $\geq$ `10 filmova`	Lista sa 10 filmova	Pretraga po nazivu vraća više od 10 filmova	$1 \to 2 \to 4 \to 5 \to 10 \to 11 \to 13 \to 15$
TC4	`izbor="2", kategorija="Drama", filmRepo < 10 filmova u kategoriji`	Lista sa svim filmovima	Filtriranje po kategoriji, manje od 10 rezultata	$1 \to 2 \to 4 \to 6 \to 10 \to 11 \to 14 \to 15$
TC5	`izbor="3", minimalnaOcjena=9.5, filmRepo = ne-prazna lista`	Prazna lista	Filtriranje po minimalnoj ocjeni ne vraća nijedan film	$1 \to 2 \to 4 \to 7 \to 10 \to 12 \to 15$
TC6	`izbor="abc", minimalnaOcjena=7.5, filmRepo = ne-prazna lista`	Prazna lista	Nevalidan unos	$1 \to 2 \to 4 \to 9 \to 15$
TC7	`izbor="1", naziv=null, filmRepo = ne-prazna lista`	Prazna lista	Test null vrijednosti naziva, pokriva null-coalescing operator	$1 \to 2 \to 4 \to 5 \to 10 \to 12 \to 15$

1.3 Sinteza rezultata White-box testiranja

Analizom metode `FiltrirajPretraziFilmove()` kroz različite kriterije white-box testiranja utvrđeno je sljedeće:

- **Obuhvat iskaza/linija (Statement Coverage):** Skup testova **TC1–TC6** omogućava izvršavanje svake linije koda barem jednom, što osigurava potpun obuhvat linija. Svaki specifičan uslov, izlaz ili grana u metodi je testiran.
- **Obuhvat odluka/grana (Decision/Branch Coverage):** Uz testove TC1-TC7, postignut je 100% obuhvat odluka. Svi ishodi odluka (TRUE i FALSE) u svim granama metode su pokriveni, uključujući rane izlaze, validne opcije pretrage i rukovanje nevalidnim unosima.
- **Obuhvat petlji (Loop Coverage):** Identificirana je jedna `foreach` petlja. Kriterij je zadovoljen primjenom strategije testiranja jednostavnih petlji (0, 1, 2, m prolaza) kroz dodatne scenarije (TC_L1–TC_L4).
- **Obuhvat toka podataka (Data Flow Coverage):** Analizirane su ključne varijable (`sviFilmovi`, `rezultati`, `izbor` itd.). Testovi pokrivaju sve kritične parove definicija i upotreba (DEF-USE), osiguravajući da podaci ispravno putuju kroz logiku filtriranja.
- **Obuhvat puteva (Path Coverage):** Sve izvedive logičke putanje kroz metodu su identificirane i testirane odgovarajućim testovima **TC1–TC7**, što potvrđuje potpunu pokrivenost puteva.

1.4 Code Tuning

Za mjerenje **vremena izvršavanja** koristi se klasa `Stopwatch` iz .NET biblioteke, koja precizno mjeri proteklo vrijeme između početka i završetka izvršenja metode. Za mjerenje **potrošnje memorije** koristi se metoda `GC.GetAllocatedBytesForCurrentThread()`, koja vraća ukupnu količinu memorije alociranu od strane tekuće niti tokom izvršavanja, čime se dobija stvarna memorijska potrošnja metode, nezavisno od učestalosti garbage collectora.

1.4.1 Korištene tehnike

U ovom poglavlju nad metodom su primijenjene tri različite tehnike code tuninga, u skladu sa kategorijama iz predavanja. Cilj je da se smanji vrijeme izvršavanja metoda, ali zadrži isto funkcionalno ponašanje.
Tehnike:

1. **Tehnika 1 - Tuning petlji**
2. **Tehnika 2 - Tuning logičkih iskaza**
3. **Tehnika 3 - Tuning izraza**

1.4.2 Tuning petlji

Prije: U originalnoj implementaciji korištena je `foreach` petlja za prolazak kroz sve filmove iz repozitorija i njihovo kopiranje u novu listu. Ovakav pristup uzrokuje dodatnu iteraciju nad podacima, povećava vrijeme izvršavanja i zahtijeva dodatnu memoriju proporcionalnu broju elemenata.

```
List<Film> sviFilmovi = new List<Film>();
foreach (var film in filmRepo.GetAll())
{
    sviFilmovi.Add(film);
}
```

Poslije: Nakon primjene tuninga, petlja je u potpunosti uklonjena jer nije imala algoritamsku vrijednost. Filmovi se obrađuju direktno kroz kolekciju vraćenu iz repozitorija, čime se eliminiše nepotrebna iteracija i dodatna potrošnja memorije.

```
List<Film> sviFilmovi = filmRepo.GetAll();



if (sviFilmovi.Count == 0)
{
    return new List<Film>();
}
```

Rezultati izvršavanja prije i poslije tuninga:

```
-----------------------------------------
REZULTATI:
-----------------------------------------
Vrijeme izvršavanja : 14148 ms
Memorija    : 6481,02 MB
-----------------------------------------
```

Slika 3: Prije tuninga

```
-----------------------------------------
REZULTATI:
-----------------------------------------
Vrijeme izvršavanja : 8755 ms
Memorija    : 1415,10 MB
-----------------------------------------
```

Slika 4: Poslije tuninga 1.

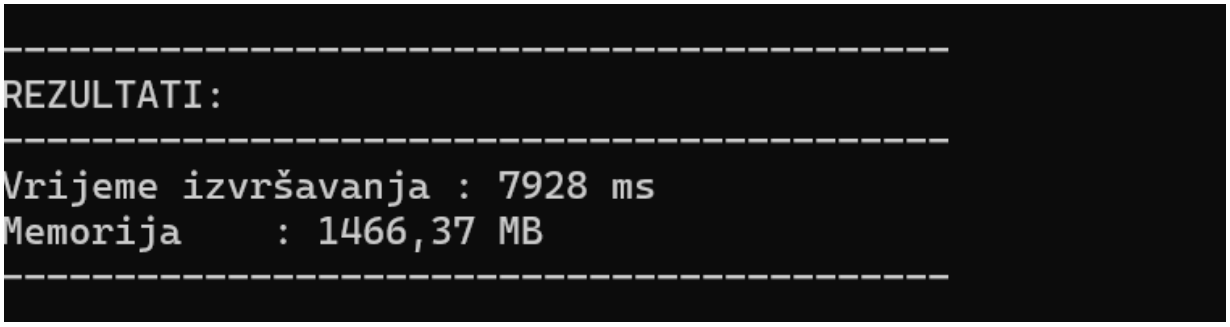
Uklanjanjem redundantne petlje za kopiranje podataka postignuto je drastične smanjenje potrošnje memorije i ubrzanje izvršavanja. Ovi rezultati potvrđuju da je eliminacija nepotrebnih alokacija ključna za performanse, čime je metoda postala znatno efikasnija i skalabilnija za rad s velikim skupovima podataka.

1.4.3 Tuning logičkih iskaza – Early Exit

Prije: Resursi se alociraju i podaci dohvataju iz baze (GetAll()) bez obzira na korisnikov unos. Čak i ako korisnik želi izaći iz aplikacije, ili je unos pogrešan, program nepotrebno troši CPU cikluse na pripremu podataka.
Poslije: U optimizovanom kodu, primijenjen je princip Guard Clauses. Uslovi za prekid (izlaz korisnika) provjeravaju se na samom početku. Ako su ispunjeni, metoda odmah završava rad (return).

```
HashSet<string> DozvoljeniIzbori = new() { "1", "2", "3" };
if (izbor == "0" || !DozvoljeniIzbori.Contains(izbor))
    return new List<Film>();
```

Rezultati izvršavanja poslije tuninga:



Slika 5: Poslije tuninga 2.

Implementacijom "Early Exit" logike vrijeme izvršavanja dodatno je smanjeno. Blagi porast memorije u drugoj fazi pripisuje se varijabilnosti *Garbage Collectora* i alokaciji `HashSet` strukture. Ova optimizacija postavlja i **čvrstu osnovu za buduće nadogradnje** (scalability). Korištenje `HashSet`-a omogućava dodavanje novih opcija pretrage u budućnosti bez degradacije performansi (kompleksnost ostaje $O(1)$), dok struktura ranog izlaza osigurava da kod ostane čitljiv i stabilan čak i ako se metoda značajno proširi novom logikom. Ukoliko korisnik odabere opciju za izlaz iz menija ili pogrešnu opciju, metoda se odmah završava - najbrži kod je onaj koji se nikad ne izvrši.

1.4.4 Tuning izraza - Optimizacija LINQ upita

Prije: Metoda kreira listu svih filmova koji zadovoljavaju kriterij pretrage (Where) prije nego što vrati rezultate. Za velike baze podataka, ovo znači da se u memoriju alocira velika lista, što povećava potrošnju memorije i vrijeme izvršavanja.if (izbor == "1")

```
        rezultati = sviFilmovi
            .Where(f => f.getNazivFilma().Contains(naziv ?? "Nepostojeci film"))
            .ToList();
    }
```

Poslije: Metoda odmah uzima samo prvih 10 rezultata pomoću Take(10) i tek tada kreira listu (ToList()). Time se značajno smanjuje broj alociranih objekata u memoriji, smanjuje se potrošnja memorije i ubrzava se izvršavanje, jer se ne prolazi kroz sve moguće rezultate.

```
if (izbor == "1")
{
    rezultati = sviFilmovi
        .Where(f => f.getNazivFilma().Contains(naziv ?? "Nepostojeci film"))
        .Take(10)
        .ToList();
}
```

```
----------------------------------------
REZULTATI:
----------------------------------------
Vrijeme izvršavanja : 2151 ms
Memorija    : 184,63 MB
----------------------------------------
```

Slika 6: Poslije tuninga 3

Rezultati ove faze nedvosmisleno potvrđuju efikasnost tuninga izraza Restrukturacijom LINQ upita – specifično, premještanjem izraza .Take(10) ispred poziva za materijalizaciju .ToList() – postignuto je značajno smanjenje izvršavanja i ušteda memorije. Pomjeranjem filtriranja (.Where) i limitiranja (.Take) prije materijalizacije (.ToList), izbjegnuto je procesiranje i alociranje stotina hiljada objekata koji nisu bili potrebni krajnjem korisniku.

1.4.5 Kodovi za Code Tuning

Kod poslije opmitizacije 1.

```
 public  List<Film> FiltrirajPretraziFilmove( string izbor,string? naziv=null,string?
 ↪  kategorija=null,double minimalnaOcjena=0)
{

    List<Film> sviFilmovi = filmRepo.GetAll();
       if (sviFilmovi.Count == 0)
    {
        return new List<Film>();
    }

    List<Film> rezultati = new List<Film>();

    if (izbor == "1")
    {
        rezultati = sviFilmovi
            .Where(f => f.getNazivFilma().Contains(naziv ?? "Nepostojeci film"))
            .ToList();
    }
    else if (izbor == "2")
    {
        rezultati = sviFilmovi
            .Where(f => f.getKategorija() == kategorija)
            .ToList();
    }
    else if (izbor == "3")
    {
        rezultati = sviFilmovi
            .Where(f => f.getOcjena() >= minimalnaOcjena)
            .ToList();
    }
    else if (izbor == "0")
    {
        return new List<Film>();
    }
    else
```

```
    {
        return new List<Film>();
    }


    if (rezultati.Any())
    {
        if (rezultati.Count > 10)
        {
            return rezultati.Take(10).ToList();
        }

        else { return rezultati; }
    }


    return new List<Film>();
}
```

Kod poslije opmitizacije 2.

```
public  List<Film> FiltrirajPretraziFilmove( string izbor,string? naziv=null,string?
↪  kategorija=null,double minimalnaOcjena=0)
{

    HashSet<string> DozvoljeniIzbori = new() { "1", "2", "3" };
    if (izbor == "0" || !DozvoljeniIzbori.Contains(izbor))
        return new List<Film>();

    List<Film> sviFilmovi = filmRepo.GetAll();

    if (sviFilmovi.Count == 0)
    {
        return new List<Film>();
    }

    List<Film> rezultati = new List<Film>();

    if (izbor == "1")
    {
        rezultati = sviFilmovi
            .Where(f => f.getNazivFilma().Contains(naziv ?? "Nepostojeci film"))
            .ToList();
    }
    else if (izbor == "2")
    {
        rezultati = sviFilmovi
            .Where(f => f.getKategorija() == kategorija)
            .ToList();
    }
    else if (izbor == "3")
    {
        rezultati = sviFilmovi
            .Where(f => f.getOcjena() >= minimalnaOcjena)
            .ToList();
    }
```

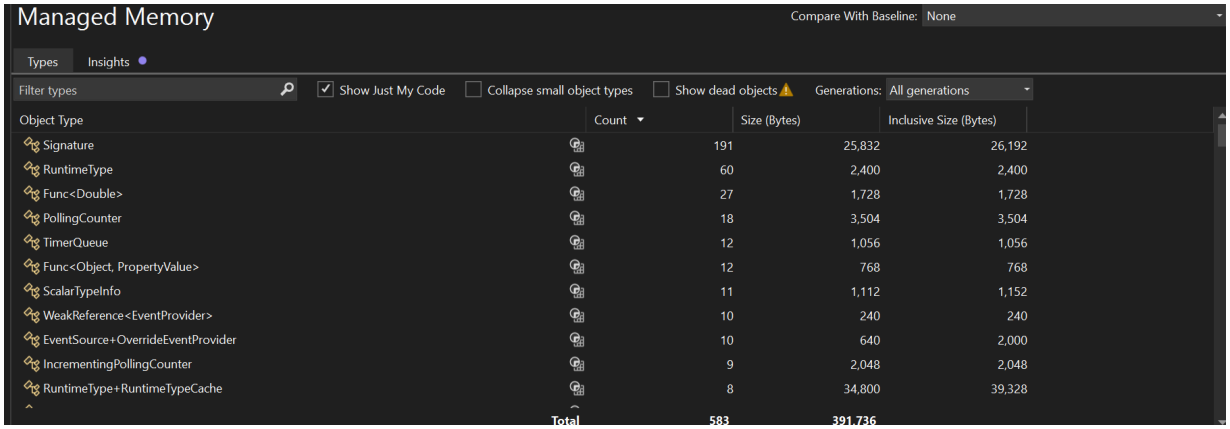
```
    if (rezultati.Any())
    {
        if (rezultati.Count > 10)
        {
            return rezultati.Take(10).ToList();
        }

        else { return rezultati; }
    }


    return new List<Film>();
}
```

Kod poslije opmitizacije 3. - Konačan kod

```
public  List<Film> FiltrirajPretraziFilmove( string izbor,string? naziv=null,string?
↪  kategorija=null,double minimalnaOcjena=0)
{

    HashSet<string> DozvoljeniIzbori = new() { "1", "2", "3" };
    if (izbor == "0" || !DozvoljeniIzbori.Contains(izbor))
        return new List<Film>();

    List<Film> sviFilmovi = filmRepo.GetAll();

    if (sviFilmovi.Count == 0)
    {
        return new List<Film>();
    }

    List<Film> rezultati = new List<Film>();

    if (izbor == "1")
    {
        rezultati = sviFilmovi
            .Where(f => f.getNazivFilma().Contains(naziv ?? "Nepostojeci film"))
            .Take(10)
            .ToList();
    }
    else if (izbor == "2")
    {
        rezultati = sviFilmovi
            .Where(f => f.getKategorija() == kategorija)
            .Take(10)
            .ToList();
    }
    else if (izbor == "3")
    {
        rezultati = sviFilmovi
            .Where(f => f.getOcjena() >= minimalnaOcjena)
            .Take(10)
            .ToList();
```

```
    }



    if (rezultati.Any())
    {
        return rezultati;
    }


    return new List<Film>();
}
```

1.4.6 Memorija



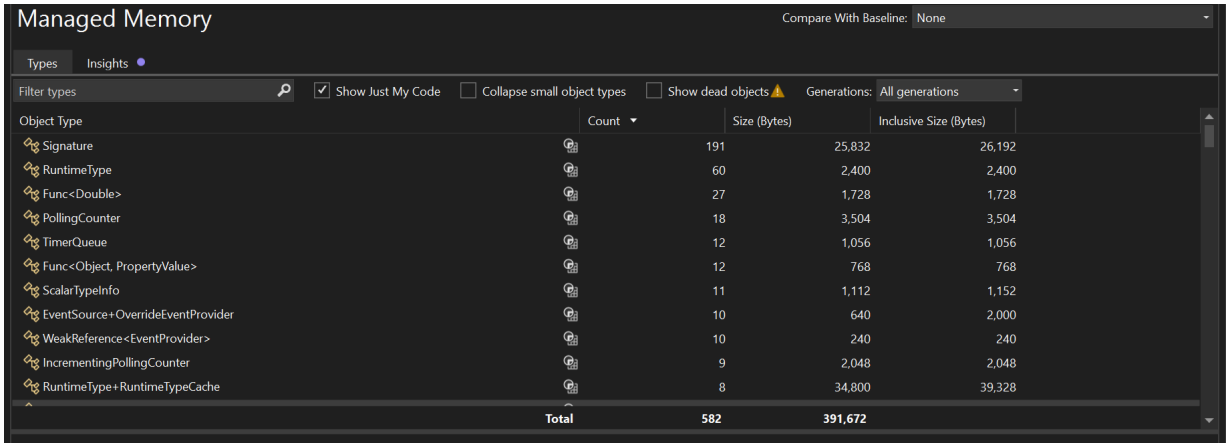
Managed Memory — Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	191	25,832	26,192
RuntimeType	60	2,400	2,400
Func<Double>	27	1,728	1,728
PollingCounter	18	3,504	3,504
TimerQueue	12	1,056	1,056
Func<Object, PropertyValue>	12	768	768
ScalarTypeInfo	11	1,112	1,152
WeakReference<EventProvider>	10	240	240
EventSource+OverrideEventProvider	10	640	2,000
IncrementingPollingCounter	9	2,048	2,048
RuntimeType+RuntimeTypeCache	8	34,800	39,328
Total	583	391,736	

Slika 7: Memorija prije



Managed Memory — Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	191	25,832	26,192
RuntimeType	60	2,400	2,400
Func<Double>	27	1,728	1,728
PollingCounter	18	3,504	3,504
TimerQueue	12	1,056	1,056
Func<Object, PropertyValue>	12	768	768
ScalarTypeInfo	11	1,112	1,152
EventSource+OverrideEventProvider	10	640	2,000
WeakReference<EventProvider>	10	240	240
IncrementingPollingCounter	9	2,048	2,048
RuntimeType+RuntimeTypeCache	8	34,800	39,328
Total	582	391,672	

Slika 8: Memorija poslije

1.4.7 Utjecaj na McCabe metriku i indeks održavanja

Verzija	V(G)	MI
Prvobitna metoda	10	48
Nakon tehnike 1 (T1)	9	49
Nakon tehnike 2 (T2)	10	48
Nakon tehnike 3 (T3)	9	50

Tablica 7: Metrike za različite verzije metode

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmMate (Debug)	83	228	2	39	1.163	415
Program	54	32	1	21	205	83
FilmMate.Services	73	149	1	25	757	272
FilmService	60	128	1	11	635	233
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	46	13		4	70	30
azurirajFilm() : void	56	7		4	39	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4
OcijeniFilmGledaoca() : void	51	10		4	57	22
FiltrirajPretraziFilmove(string, string?, string?, dou	50	9		7	48	21
filtrirajPoNazivu(List<Film>) : List<Film>	66	4		6	16	7

Slika 9: Rezultati alata - konačni kod

Analizom metrika kroz faze refaktorisanja može se uočiti postepen utjecaj primijenjenih tehnika na složenost i održivost koda.

- **Nakon primjene tehnike 1 (T1):** Prva tehnika refaktorisanja rezultirala je blagim smanjenjem ciklomatske složenosti sa 10 na 9, dok je indeks održivosti porastao sa 48 na 49. Ovo poboljšanje je posljedica djelimičnog pojednostavljivanja kontrolnog toka (uklanjanje suvišne foreach petlje).
- **Nakon primjene tehnike 2 (T2):** Nakon druge tehnike, vrijednosti metrika su se vratile na početne vrijednosti (V(G) = 10, MI = 48). Ovaj rezultat ukazuje da primijenjene izmjene nisu imale pozitivan efekat na složenost koda, ali su imale dobar uticaj na performanse. Iako su određeni dijelovi koda reorganizirani, uvođenje *Early Exit* strategije dodalo je novi uslov na početku metode.
- **Nakon primjene tehnike 3 (T3) – konačna verzija:** Nakon provedene optimizacije, zabilježene su sljedeće vrijednosti metrika:
  - **Ciklomatska složenost (McCabe) = 9:** Uklanjanjem završne provjere (`if (rezultati.Count > 10)`)- upotreba .Take(10) učinila je tu eksplicitnu provjeru nepotrebnom - je uklonjena jedna grana koda. Vrijednost 9 je i dalje relativno visoka (blizu granice od 10), prvenstveno zbog preostalih `if/else if` blokova za odabir filtera.
  - **Indeks održivosti (Maintainability Index – MI) = 50:** Indeks održivosti bilježi blagi porast sa 48 na 50. Iako je uočen napredak u čitljivosti i strukturi koda, vrijednost MI i dalje ostaje u zoni upozorenja (ispod 65), što ukazuje na potrebu za dodatnim refaktorisanjima radi dugoročne održivosti.

Zaključak: Finalizacijom tuning faze postignuta su drastična poboljšanja performansi: vremena izvršavanja i memorije. Međutim, statička analiza ukazuje na to da optimizacija resursa nije značajno unaprijedila strukturu koda. Zbog prisustva duplikacije logike i smanjene čitljivosti, postoji jasan prostor i potreba za **Refaktoringom**.

1.5 Refaktoring

Nakon izvršenog code tuninga nad metodom napravljen je dodatni korak refaktorisanja sa ciljem da se dodatno poboljšaju vrijednosti metrika, bez promjene funkcionalnosti metode.

Analizom je utvrđeno da je originalna metoda kršila princip pojedinačne odgovornosti (Single Responsibility Principle - SRP) jer je istovremeno obavljala validaciju, dohvat podataka, logiku grananja i filtriranje.

Kao rješenje, primijenjena je strategija **dekompozicije**. Umjesto premještanja kompleksnosti u jednu pomoćnu metodu, proces je razbijen na vertikalni lanac odgovornosti:

1. **Validacijski sloj (Javna metoda):** Zadužen isključivo za provjeru ulaznih parametara i poziv filtriranja.
2. **Podatkovni sloj :** Zadužen za dohvat podataka i provjeru praznih kolekcija.
3. **Logički sloj:** Zadužen za odabir strategije filtriranja (putem `switch` izraza).
4. **Izvršni sloj :** Zadužen za konkretnu implementaciju filtera.

Ovim pristupom postignuto je da niti jedna metoda u lancu nema visoku složenost, čime je MI maksimiziran.

1.5.1 Primijenjene stavke iz check liste

U Tabeli 8 prikazan je pregled primijenjenih tehnika refaktoringa mapiranih na specifične stavke iz Check liste.

Kategorija	Stavka iz Check liste	Implementacija / Obrazloženje
Strukture	Da li su neki moduli pretjerano kompleksni i trebaju li se restrukturirati ili razdvojiti u više funkcija?	**Da.** Metoda je restrukturirana tehnikom *Extract Method*. Kreiran je lanac manjih metoda kako bi se smanjila ciklomatska složenost glavne metode.
Petlje / grananje	Jesu li svi slučajevi pokriveni u IF-ELSEIF ili CASE blokovima?	**Da.** Logika grananja je izmještena u zasebnu metodu (`SelektujFiltriranje`) koja koristi `switch` sa obaveznom `default` granom za pokrivanje svih slučajeva.
Strukture	Da li se koriste simbolička imena umjesto 'magičnih brojeva' ili string konstanti?	**Da.** Literali ("1", "2", "3") i magični brojevi (10) zamijenjeni su imenovanim konstantama (`OPCIJA_NAZIV`, `MAX_REZULTATA`), što poboljšava čitljivost.
Defenzivno programiranje	Jesu li ulazni argumenti testirani na validnost?	**Da.** Javna metoda sadrži *Guard Clause* (`!JeValidanIzbor`) koji odmah prekida izvršavanje ako su argumenti neispravni.
Defenzivno programiranje	Jesu li uključene provjere praznih kolekcija?	**Da.** U metodi (`DohvatiIPrijemniFilter`) ima provjera `Count == 0` koja sprječava nepotrebno procesiranje praznih listi.

Tablica 8: Mapiranje refaktoringa na stavke iz check liste

Uvođenje simboličkih konstanti

Originalni kod je koristio magične vrijednosti poput stringova ("1", "2", "3") i numeričke konstante (10) direktno u logici metode. Radi poboljšanja čitljivosti i održivosti, ove vrijednosti su zamijenjene imenovanim konstantama.

```
private const int MAX_REZULTATA = 10;
private const string OPCIJA_IZLAZAK = "0";
private const string OPCIJA_NAZIV = "1";
private const string OPCIJA_KATEGORIJA = "2";
private const string OPCIJA_OCJENA = "3";
```

Validacija ulaznih parametara (Guard Clause)

Radi defenzivnog programiranja, uvedena je posebna metoda za validaciju ulaznog parametra `izbor`. Neispravni ulazi se odbacuju odmah, čime se izbjegava nepotrebno izvršavanje ostatka metode.

```
private bool JeValidanIzbor(string izbor) =>
    izbor is OPCIJA_NAZIV or OPCIJA_KATEGORIJA or OPCIJA_OCJENA or OPCIJA_IZLAZAK;
```

Primjena tehnike *Extract Method*

Originalna metoda je objedinjavala više odgovornosti: validaciju, dohvat podataka, grananje i filtriranje. Primjenom tehnike *Extract Method*, logika je razbijena u više manjih metoda, od kojih svaka ima jasno definisanu odgovornost.

```
public virtual List<Film> FiltrirajPretraziFilmove(
    string izbor, string? naziv = null,
    string? kategorija = null, double minimalnaOcjena = 0)
{
    if (!JeValidanIzbor(izbor))
        return new List<Film>();

    return DohvatiIPrijemniFilter(izbor, naziv, kategorija, minimalnaOcjena);
}
```

Provjera praznih kolekcija

U metodi `DohvatiIPrijemniFilter` uvedena je provjera `sviFilmovi.Count == 0`, koja sprječava izvršavanje filtriranja nad praznom listom. Ova provjera smanjuje broj mogućih putanja izvršavanja i doprinosi robusnosti sistema.

```
var sviFilmovi = filmRepo.GetAll();


    if (sviFilmovi.Count == 0)
    {
        return new List<Film>();
    }
```

Pojednostavljenje grananja korištenjem `switch` izraza

Višestruki `if--else if` blokovi zamijenjeni su modernim `switch` izrazom, čime je smanjen broj eksplicitnih grananja i povećana čitljivost koda.

```
private IEnumerable<Film> SelektujFiltriranje(
    List<Film> filmovi, string izbor,
    string? naziv, string? kat, double ocjena) =>
    izbor switch
    {
        OPCIJA_NAZIV      => FiltrirajPoNazivu(filmovi, naziv),
        OPCIJA_KATEGORIJA => FiltrirajPoKategoriji(filmovi, kat),
        OPCIJA_OCJENA     => FiltrirajPoOcjeni(filmovi, ocjena),
        OPCIJA_IZLAZAK    => Enumerable.Empty<Film>()
    };
```

1.5.2 Refaktorisani kod

Ispod je prikazan konačni kod nakon refaktoringa, koji demonstrira jasnu podjelu odgovornosti. Prikazani su kodovi za svaku dodanu funkciju.

```
private const int MAX_REZULTATA = 10;
private const string OPCIJA_IZLAZAK = "0";
private const string OPCIJA_NAZIV = "1";
private const string OPCIJA_KATEGORIJA = "2";
private const string OPCIJA_OCJENA = "3";


public virtual List<Film> FiltrirajPretraziFilmove(string izbor, string? naziv = null, string?
↪ kategorija = null, double minimalnaOcjena = 0)
{
    if (!JeValidanIzbor(izbor))
    {
        return new List<Film>();
    }

    //primjena filtera
    return DohvatiIPrijemniFilter(izbor, naziv, kategorija, minimalnaOcjena);
}

// Odgovornost: Rad sa bazom i provjera prazne liste.
private List<Film> DohvatiIPrijemniFilter(string izbor, string? naziv, string? kategorija, double
↪ minimalnaOcjena)
{
    var sviFilmovi = filmRepo.GetAll();


    if (sviFilmovi.Count == 0)
    {
        return new List<Film>();
    }

    var filtrirani = SelektujFiltriranje(sviFilmovi, izbor, naziv, kategorija, minimalnaOcjena);

    return filtrirani.Take(MAX_REZULTATA).ToList();
}

// Odgovornost: Odabir logike filtriranja.
private IEnumerable<Film> SelektujFiltriranje(List<Film> filmovi, string izbor, string? naziv,
↪ string? kat, double ocjena) =>
    izbor switch
    {
        OPCIJA_NAZIV      => FiltrirajPoNazivu(filmovi, naziv),
        OPCIJA_KATEGORIJA => FiltrirajPoKategoriji(filmovi, kat),
        OPCIJA_OCJENA     => FiltrirajPoOcjeni(filmovi, ocjena),
        OPCIJA_IZLAZAK    => Enumerable.Empty<Film>()
    };

// Filtriranje
private IEnumerable<Film> FiltrirajPoNazivu(IEnumerable<Film> filmovi, string? naziv) =>
    filmovi.Where(f => f.getNazivFilma().Contains(naziv ?? string.Empty));

private IEnumerable<Film> FiltrirajPoKategoriji(IEnumerable<Film> filmovi, string? kat) =>
```

```
    filmovi.Where(f => f.getKategorija() == kat);

private IEnumerable<Film> FiltrirajPoOcjeni(IEnumerable<Film> filmovi, double ocjena) =>
    filmovi.Where(f => f.getOcjena() >= ocjena);

// Pomocna validacija
private bool JeValidanIzbor(string izbor) =>
    izbor is OPCIJA_NAZIV or OPCIJA_KATEGORIJA or OPCIJA_OCJENA or OPCIJA_IZLAZAK;
```

1.5.3 Utjecaj refaktorisanja na metrike

Utjecaj pojedinačnih refaktoring tehnika na metrike može se sažeti na sljedeći način:

- **Extract Method:** Smanjio ciklomatsku složenost glavne metode razdvajanjem grananja u zasebne, linearne metode.
- **Guard Clause:** Eliminisao potrebu za dodatnim ugniježđenim granama, čime je smanjen broj logičkih odluka.
- **Switch izraz umjesto IF–ELSEIF lanca:** Pojednostavio logiku odabira i poboljšao čitljivost, uz bolju kontrolu svih mogućih slučajeva.
- **Imenovane konstante:** Ne utiču direktno na $V(G)$, ali značajno poboljšavaju MI kroz povećanu čitljivost i smanjenu kognitivnu složenost.

U prilogu se nalaze rezultati iz Code Metrics alata, nakon refaktoringa.

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmService	64	129	1	10	648	233
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	46	13		4	70	30
azurirajFilm() : void	56	7		4	39	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4
OcijeniFilmGledaoca() : void	51	10		4	57	22
MAX_REZULTATA : int	93	0		0	1	1
OPCIJA_NAZIV : string	93	0		0	1	1
OPCIJA_KATEGORIJA : string	93	0		0	1	1
OPCIJA_OCJENA : string	93	0		0	1	1
FiltrirajPretraziFilmove(string, string?, string?, dou	67	2		2	10	6
DohvatiIPrijemniFilter(string, string?, string?, doub	70	2		5	17	5
SelektujFiltriranje(List<Film>, string, string?, string	87	1		4	11	1
FiltrirajPoNazivu(IEnumerable<Film>, string?) : IEn	82	2		4	4	2
FiltrirajPoKategoriji(IEnumerable<Film>, string?) : I	84	1		4	2	2
FiltrirajPoOcjeni(IEnumerable<Film>, double) : IEn	84	1		4	2	2
JeValidanIzbor(string) : bool	93	1		0	3	1

Slika 10: Rezultati alata

Glavna metoda `FiltrirajPretraziFilmove`, koja je inicijalno imala nizak indeks održivosti, sada ostvaruje **Maintainability Index (MI) od 67** i **Ciklomatsku složenost ($V(G)$) od 2**.

- **Složenost:** Programi (moduli) sa $V(G)$ manjim od 5 su generalno jednostavniji i razumljiviji. S obzirom da je naša metoda svedena na $V(G) = 2$, ona spada u kategoriju trivijalno jednostavnih modula za razumijevanje.
- **Održivost:** Moduli sa MI vrijednostima u rasponu 65–85 klasificiraju se kao *srednje dobri za održavanje*. Naš rezultat od 67 uspješno prelazi donji prag (65), čime je kod iz zone rizika prešao u zonu prihvatljive održivosti.

Pored glavne metode, analiza pokazuje da su i novokreirane pomoćne metode visoko održive (MI > 70), što potvrđuje da kompleksnost nije sakrivena, već efektivno eliminisana.

2 Metoda DodajFilm() - Rubina Rekić

U ovoj sekciji bit će izvršena analiza metode DodajFilm() iz FilmService kalse.
Pomoću Code Metircs alata utvrđeno je da ova metoda ima ciklomatsku složenost koja zadovoljava uvjete postavljene u zatatku, pa je upravo iz toga razloga ona izabrana za analizu.

2.1 Opis metode

Odabrana metoda omogućava korisniku da doda novi film u repozitorij (filmRepo). Osim toga što omogućava unos, metoda također radi i validaciju i perzistenciju (čuvanje) unesenih podataka.
Ispod se nalazi kod metode, kako bismo se mogli referencirati na neke njene dijelove kasnije pri analizi.
baselinestretchbaselinestretch

```
        public void dodajFilm()
        {
            Console.Write("Naziv: ");
            string naziv = Console.ReadLine() ?? "";
            if (string.IsNullOrWhiteSpace(naziv) || naziv.Length < 2)
            {
                Console.WriteLine("Naziv mora imati bar 2 karaktera!");
                return; }
            bool postoji = false;
            foreach (var f in filmRepo.GetAll())
            {
                if (f.getNazivFilma().ToLower() == naziv.ToLower())
                {
                    postoji = true;
                    break;}}

            if (postoji)
            {
                Console.WriteLine("Film već postoji!");
                return;}

            Console.Write("Kategorija: ");
            string kat = Console.ReadLine() ?? "";
            Console.Write("Ocjena (1-10): ");
            double ocjena = 0;
            bool validOcjena = double.TryParse(Console.ReadLine(), out ocjena);

            if (validOcjena)
            {
                if (ocjena < 1 || ocjena > 10)
                {
                    ocjena = 5;
                    Console.WriteLine("Neispravna ocjena, postavljena na 5.");} }
            else
            {
                ocjena = 5;
                Console.WriteLine("Neispravan format ocjene, postavljena na 5.");}

            Console.Write("Godina: ");
            int godina = int.TryParse(Console.ReadLine(), out godina) ? godina : 0;
            filmRepo.GetAll().Add(new Film(naziv, kat, ocjena, godina));
            filmRepo.Sacuvaj();
            Console.WriteLine("Film uspješno dodat!"); }
```

2.2 McCabe metrika

2.2.1 Proračun McCabe metrike

Formula koja će biti primjenjena za računanje McCabe metrike je sljedeća:

$$V(G) = e - n + 2p$$

Značenje parametara je sljedeće:

1. **e** je broj ivica(tokova) u grafu programskog toka,
2. **n** je broj čvorova u grafu programskog toka,
3. **p** je broj formiranih neovisnih dijelova grafa.

Konkretno, u kontekstu metode koja se analizira, imamo da je:

1. **p=1** jer je u pitanju analiza jedne metode,
2. **n=11** čvorova u kontrolnom grafu,
3. **e=21** usmjerenih grana u kontrolnom grafu

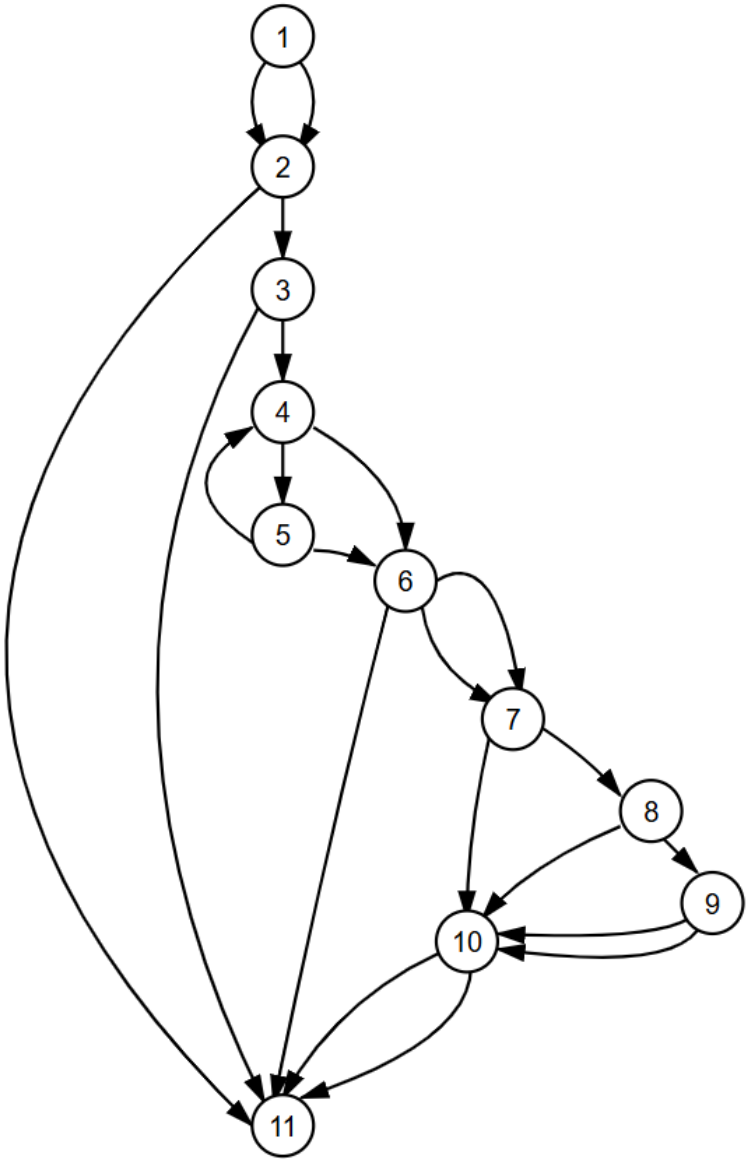
Uvrštavanjem u formulu dobijamo:

$$V(G) = 21 - 11 + 2 * 1$$

$$V(G) = 12 \tag{2}$$

2.2.2 Kontrolni graf

U cilju validacije urađenog proračuna ciklomatske složenosti, na slici ispod je kontrolni graf. Na osnovu ovog grafa izvršena je identifikacija 11 čvorova i 20 ivica, što služi kao osnova za primjenu McCabeove formule.



Slika 11: Kontrolni dijagram

2.2.3 Poređenje rezultata

Nakon manuelnog proračuna, izvršena je analiza pomoću alata *Code Metrics* u Visual Studio. Rezultati su prikazani u nastavku:

Code Metrics Results

Filter: None | Min: | Max:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmMate (Debug)	81	220	2	33	969	344
Program	59	27	1	13	79	31
SortirajMeni	52	14	1	3	44	19
FilmMate.Data	88	22	1	11	98	32
FilmMate.Models	95	26	2	9	103	29
FilmMate.Services	73	131	1	24	645	233
FilmService	59	110	1	10	523	194
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	46	13		4	70	30
azurirajFilm() : void	56	7		4	39	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4

Slika 12: Code Metrics

Na osnovu slike, vidimo da se izračunata ciklomatska kompleksnost poklapa sa prethodno izračunatom.

Kvalitet koda: Vrijednost $V(G) = 12$ spada u opseg 10–20, što se prema McCabeovoj skali i Jonesovim istraživanjima smatra umjerenim rizikom i alarmom za pojačanu kontrolu. Budući da prelazi preporučenu granicu jednostavnosti (< 10), metoda se klasifikuje kao kompleksna. Osnovni razlog ovom leži u tome što metoda krši princip pojedinačne odgvornosti (SRP). Naime, metoda vrši interakciju s konzolom, validaciju poslovnih pravila, parsiranje podataka i perzistenciju u repozitorij.

Poteškoće u testiranju: Ciklomatska složenost direktno definiše broj nezavisnih putanja koje je potrebno testirati. Vrijednost $V(G) = 12$ znači da je potrebno pripremiti najmanje 12 jedinstvenih testnih slučajeva kako bi se postigla potpuna pokrivenost grana (branch coverage).
Dodatnu poteškoću predstavlja direktna ovisnost o Console klasi, što onemogućava automatizovano jedinično testiranje bez manuelnog unosa ili kompleksnog preusmjeravanja standardnog ulaza/izlaza.

Indeks održivosti (MI = 48) : Metoda se sa indeksom održivosti od 48 nalazi u "žutoj zoni" (ispod 65), što je klasifikuje kao metodu otežanu za održavanje. Ovako niska vrijednost je rezultat kombinacije visoke ciklomatske složenosti i broja linija koda (56 linija izvornog koda).
Svaka promjena u logici validacije ili načinu unosa podataka zahtijeva modifikaciju ove metode, što povećava rizik od uvođenja regresijskih grešaka.

2.3 White-box testiranje

U ovoj sekciji bit će provedeno White-box testiranje, kako bi se ispitala interna struktura i logika odabrane metode, te otkrile postojeće greške. Budući da metoda ima ciklomatsku složenost 12, ona je idealan kandidat za White-box testiranje.
Isto tako, obzirom na njenu strukturu, gotovo sve tehnike White-box testiranja obrađene na predavanjima mogu se primijeniti, uključujući obuhvat iskaza, obuhvat grana/odluka, obuhvat uslova, obuhvat petlji i obuhvat toka podataka.
Tehnika MCDC također je primjenjiva, ali je složenija i manje praktična za ovu metodu. Potpuni obuhvat svih mogućih puteva izvršavanja nije praktično izvediv zbog velikog broja kombinacija, stoga se će fokus testiranja biti stavljen na linearno nezavisne puteve.

2.3.1 Obuhvat iskaza/linija

Ova testna strategija ima težnju da dizajnira testne slučajeve tako da se svaki iskaz, tj. svaka linija koda izvrši barem jednom.
Cilj je za da postignemo 100% pokrivenost iskaza uz *minimalnan broj testnih slučajeva.*
Za DodajFim metodu potrebna su minimalno 5 testnih slučajeva kako bismo bili sigurni da se svaka linija izvršila barem jednom.

U tabeli ispod prikazani su ti testni scenariji:

	Opis scenarija	**Ulazni podaci (Konzola)**	**Očekivani ishod**
TC1	Nevalidan naziv (prekratak unos)	`naziv = "A"`	Ispis greške i prekid (linije 5-8).
TC2	Unos postojećeg filma (duplikat)	`naziv = "Inception"` (film već postoji)	Izvršavanje petlje i prekid (linije 10-20).
TC3	Validni podaci (Happy Path)	`naziv = "Titanic"`, `ocjena = "9"`	Uspješno dodavanje filma.
TC4	Neispravan format ocjene (tekst)	`naziv = "Avatar"`, `ocjena = "odličan"`	Ulazak u `else` blok (34-37 linije).
TC5	Ocjena izvan opsega (broj veći 10)	`naziv = "Matrix"`, `ocjena = "15"`	Ulazak u unutrašnji `if` blok (30-33 blok).

Tablica 9: Testni slučajevi za obuhvat iskaza

Kombinacijom ovih pet slučajeva osigurano je da su sve linije koda, uključujući i one unutar ugniježđenih `if` blokova za validaciju vrijednosti ocjene, u izvršene.

2.3.2 Obuhvat odluka/grana

U ovoj sekciji fokus je na tačkama grananja u kodu. Cilj je osigurati da svaka odluka u programu barem jednom rezultuje vrijednošću True i barem jednom vrijednošću False.
U metodi dodajFilm() imamo tačno deset ključnih tačaka odluke, što možemo i direktno očitati sa kontrolnog grafa iz prve sekcije.

1. **Null-coalescing operator (`??`) za naziv**: Provjera da li je `Console.ReadLine()` vratio `null`.
2. **Validacija naziva (`if`)**: Provjera složenog uslova `IsNullOrWhiteSpace` ili da li je dužina naziva manja od 2 karaktera.
3. **Kontrola petlje (`foreach`)**: Odluka o ulasku u tijelo petlje (ako lista nije prazna) ili nastavku koda (ako je lista prazna ili su svi elementi obiđeni).
4. **Pronađen duplikat (`if uslov` unutar petlje)**: Provjera da li se naziv unesenog filma podudara sa postojećim filmom u bazi.
5. **Provjera zastavice (`if (postoji)`)**: Odluka o prekidu metode ako je film već pronađen ili nastavku unosa ako nije.
6. **Null-coalescing operator (`??`) za kategoriju**: Provjera unosa za kategoriju i dodjela praznog stringa u slučaju `null` vrijednosti.
7. **Rezultat parsiranja (`double.TryParse`)**: Implicitna odluka koja određuje vrijednost logičke varijable `validOcjena`.
8. **Validnost formata ocjene (`if (validOcjena)`)**: Grananje na osnovu toga da li je korisnik unio brojčanu vrijednost ili neispravan format.
9. **Provjera ranga ocjene (`if (ocjena < 1 || ocjena > 10)`)**: da li je unesena ocjena unutar dozvoljenog opsega.
10. **Ternarni operator (`? :`) za godinu**: Grananje pri dodjeli vrijednosti varijabli `godina` na osnovu uspješnosti parsiranja.

Kontrolni graf nam pomaže u identifikaciji svih putanja kroz metodu, tj. svih slučajeva koje moramo ovdje testirati.
Za potpuni obuhvat grana, moramo "proći" kroz svaku granu (strelicu) na grafu barem jednom.

Ispod je navedeno 5 testnih slučajeva, u kojima su sadržani svi mogući ishodi za svaki od uslova. Obzirom da iz čvora 6 imamo tri moguća puta (2 su vezana za ?? operator, jedan za return ako film vec postoji), u testnim slučajevima su navedena sva tri, nazvani kao G1, G2 i G3.

Tablica 10: Minimalni set testova za 100% obuhvat grana

ID	Ulazni podaci i Preuslov	Putanja na grafu	Pokrivene odluke (Ishodi)
TC1	`naziv = null`	1-2-11	**1(T), 2(T)**
TC2	`naziv = "Avatar"` (Baza: `["Avatar"]`)	1-2-3-4-5-6-11	1(F), 2(F), **3(T), 4(T), 5(T), 6(G1)**
TC3	`naziv = "Joker"`, `kat = null`, `ocjena = "15"`, `godina = "abc"` (Baza: `["Seven"]`)	1-2-3-4-6-7-8-9-10-11	**4(F)**, 5(F), **6(G2)**, **7(T), 8(T), 9(T), 10(F)**
TC4	`naziv = "Matrix"`, `kat = "Akcija"`, `ocjena = "8"`, `godina = "2024"` (Baza: `[]`)	1-2-3-6-7-8-9-10-11	**3(F)**, **6(G3)**, **9(F), 10(T)**
TC5	`naziv = "Alien"`, `ocjena = "xyz"`	1-2-3-6-7-10-11	**7(F), 8(F)**

2.3.3 Obuhvat uslova

Pokrivenost uslova(condition coverage) je tehnika testiranja koja procjenjuje svaki pojedinačni logički uslov unutar složenih izraza u iskazima selekcije i petljama.

U metodi DodajFilm() postoje dva ovakva uslova koja će biti potrebno testirati:

```
            if (string.IsNullOrWhiteSpace(naziv) || naziv.Length < 2)
```

```
          if (ocjena < 1 || ocjena > 10)
```

U nastavku svaki složeni izraz razbijamo na njegove osnovne dijelove. Da bismo postigli potpun obuhvat uslova (*Condition Coverage*), neophodno je osigurati da svaki od ovih pod-uslova barem jednom poprimi vrijednost **True (T)** i barem jednom vrijednost **False (F)**.

- **Složeni uslov 1:**

  C_1: *string.IsNullOrWhiteSpace(naziv)*

  C_2: $naziv.Length < 2$

- **Složeni uslov 2:**

  C_3: $ocjena < 1$

  C_4: $ocjena > 10$

U tabeli ispod nalaze se svi potrebni testni slučajevi za 100% obuhvat uslova:

Tablica 11: Testni slučajevi za obuhvat uslova

ID	**Ulazni podaci**	C_1	C_2	C_3	C_4	**Cilj testa**
TC1	`naziv = ""`	**T**	-	-	-	Testira $C_1 = T$.
TC2	`naziv = "A"`	**F**	**T**	-	-	Testira $C_2 = T$.
TC3	`naziv = "Matrix"`	**F**	**F**	-	-	Testira $C_1 = F, C_2 = F$.
TC4	`ocjena = 0.5`	-	-	**T**	**F**	Testira $ocjena < 1$.
TC5	`ocjena = 15.0`	-	-	**F**	**T**	Testira $ocjena > 10$.

2.3.4 Obuhvat petlji

Za metodu dodajFilm(), jedina petlja je foreach koja prolazi kroz listu filmova kako bi provjerila postoji li duplikat.
U analizi obuhvata petlji (*Loop coverage*), primjenjujemo sistematizovanu strategiju testiranja jednostavne petlje kroz pet ključnih scenarija:

- **0 prolaza:** Preskakanje unutrašnjosti petlje u potpunosti.
- **1 prolaz:** Izvršavanje tijela petlje tačno jednom. item **2 prolaza:** Izvršavanje tijela petlje dva puta kako bi se provjerila stabilnost iteracije.
- **m prolaza:** Slučajan broj prolaza kroz petlju, pri čemu je $m < n$.
- **Granični slučajevi:** Testiranje sa $n - 1$ i n prolazaka kroz petlju.

Tabela testnih slučajeva nalazi se u nastavku:

Tablica 12: Testni slučajevi za obuhvat petlji (Loop coverage)

ID	**Scenario (n)**	**Putanja na grafu**	**Opis i očekivani ishod**
TC1	0 prolaza (Prazna lista)	$4 \to 6$	Kolekcija je prazna. Petlja se preskače, a varijabla `postoji` ostaje `false`.
TC2	1 prolaz (Bez podudaranja)	$4 \to 5 \to 4 \to 6$	Lista sadrži jedan film koji nije duplikat. Program jednom izvršava provjeru u čvoru 5.
TC3	2 prolaza (Bez podudaranja)	$4 \to 5 \to 4 \to 5 \to 4 \to 6$	Lista sadrži dva filma, nijedan nije duplikat. Provjerava se stabilnost prelaza između iteracija.
TC4	m prolaza ($m < n$)	$4 \to (\dots) \to 5 \to 4 \to 6$	Testiranje sa prosječnim brojem elemenata u listi bez pronalaska duplikata.
TC5	Raniji izlaz (`break`)	$4 \to 5 \to 6$	Pronalazak duplikata u bilo kojoj iteraciji. Iz čvora 5 se putem `break` komande direktno ide u čvor 6.

2.3.5 Obuhvat toka podataka

U sekciji Obuhvat toka podataka (Data Flow Coverage), fokus se pomjera sa samih putanja u grafu na životni ciklus varijabli unutar koda.
Treba za svaku varijablu biti pokrivena putanja od njene definicije (DEF) do mjesta upotrebe (USE).
Ova tehnika je izuzetno pogodna za dijelove koda sa ugniježđenim `if` iskazima i petljama, kao što je metoda `dodajFilm()`, jer omogućava precizno praćenje toka varijabli. Primjenom ove metode mogu se otkriti greške poput neinicijaliziranih ili pogrešno inicijaliziranih varijabli.

Varijabla	**Definicija (linija)**	**Upotreba (linija)**	**Opis/Tip upotrebe**
`naziv`	4	5, 12, 41	Validacija dužine, poređenje u petlji i prosljeđivanje konstruktoru.
`postoji`	9	14, 17	Inicijalizacija i provjera zastavice za duplikat.
`ocjena`	25	30, 32, 36, 41	Parsiranje, validacija ranga i finalni upis u objekat.
`validOcjena`	26	28	Određivanje toka izvršavanja na osnovu formata unosa.
`godina`	40	41	Parsiranje i dodjela pri instanciranju filma.

Tablica 13: Analiza definicija i upotreba (def-use) ključnih varijabli

Validacija putanja kroz testove:

Svi identifikovani DEF-USE parovi su pokriveni testnim scenarijima koji su prethodno definisani u sekciji *Obuhvat iskaza* (Tabela 9), čime je potvrđen integritet podataka kroz sve grane izvršavanja:

- Varijabla `naziv` prati putanju od unosa do finalne perzistencije kroz scenarije nevalidne dužine (**TC1**), provjere duplikata (**TC2**) i uspješnog dodavanja (**TC3**).
- Zastavica `postoji` testirana je u oba stanja: ishod *True* je pokriven scenarijem unosa duplikata (**TC2**), dok je ishod *False* pokriven scenarijem „Happy Path" (**TC3**).
- Varijabla `ocjena` prolazi kroz sve tri identifikovane definicije: direktni validni unos (**TC3**), re-definicija na vrijednost 5 zbog neispravnog formata (**TC4**) ili zbog unosa izvan opsega (**TC5**).
- Kontrolne varijable `validOcjena` i `godina` su testirane kroz scenarije ispravnog unosa (**TC3**) i različite formate pogrešnih unosa (**TC4** i **TC5**), čime su pokrivene sve putanje.

2.3.6 Obuhvat puteva

Obuhvat puteva (Path Coverage) je najstroži kriterij White-box testiranja. Za razliku od obuhvata grana (gdje svaka strelica mora biti prođena bar jednom), obuhvat puteva zahtijeva da se testiraju sve moguće kombinacije grana od početka do kraja metode.
Za metodu koju analiziram, ovo je malo nezgodno, obzirom da, zbog prisustva petlje, broj teorijskih puteva može biti jako velik.
Ipak, za svrhu izrade zadatka, u ovom dijlu fokus je stavljen isključivo na nezavisne puteve. Tj. puteve analiziramo kroz scenario nula ili scenario jedne iteracije kako bismo identifikovali osnovne logičke pravce kretanja kroz program.

Tablica 14: Identifikacija nezavisnih puteva kroz metodu dodajFilm()

Putanja	**Čvorovi na grafu**
P1	$1 \to 2 \to 11$
P2	$1 \to 2 \to 3 \to 11$
P3	$1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 11$
P4	$1 \to 2 \to 3 \to 4 \to 6 \to 7 \to 10 \to 11$
P5	$1 \to 2 \to 3 \to 4 \to 6 \to 7 \to 8 \to 10 \to 11$
P6	$1 \to 2 \to 3 \to 4 \to 6 \to 7 \to 8 \to 9 \to 10 \to 11$

2.4 Code tuning

Code Tuning (podešavanje koda) je praksa promjene-modifikacije konkretnog koda na način da se poboljšaju njegove performanse.

2.4.1 Problem pri tuningu i rješenje

Osnovni problem na koji sam naišla radeći code tuning bio je što metoda DodajFilm() očekuje unos iz konzole. Za potrebe testiranja performansi, gdje je trebalo unositi 10000 filmova, ovakav način je neprihvatljiv za testiranje. Zato sam napravila jednu metodu DodajFilmTest, koja je u potpunosti identična mojoj analiziranoj DodajFilm metodi, tj zadrzava svu logiku i kompleksnost, ali je izmijenjena na način da direktno prima potrebne parametre, umjesto da radi sa konzolom.
Sve izmjene pri testiranju i izračuni rađeni su na ovoj metodi. Kod se nalazi ispod:

```
public void dodajFilmTest(string naziv, string katUnos, string ocjenaUnos, string
↪ godinaUnos)
{

    if (string.IsNullOrWhiteSpace(naziv) || naziv.Length < 2)
    {
        return;
    }
    bool postoji = false;
    foreach (var f in filmRepo.GetAll())
    {
        if (f.getNazivFilma().ToLower() == naziv.ToLower())
        {
            postoji = true;
            break;
        }
    }

    if (postoji)
    {
        return;
    }

    double ocjena = 0;
    bool validOcjena = double.TryParse(ocjenaUnos, out ocjena);

    if (validOcjena)
    {
        if (ocjena < 1 || ocjena > 10)
        {
            ocjena = 5;
        }
    }
    else
    {
        ocjena = 5;
    }
    int godina = int.TryParse(godinaUnos, out godina) ? godina : 0;

    filmRepo.GetAll().Add(new Film(naziv, katUnos, ocjena, godina));
    filmRepo.Sacuvaj();
}
```

Nakon uvođenja potrebene metode, možemo preći direktno na Tuning. Uz napomenu da svaka izmjena ispod opisana rađena je kako na pomočnoj, tako uporedo i na naliziranoj metodi, samo je u svrhu tetsiranja postignutih rezultata korištena napravljena pomoćna metoda.

2.4.2 Izmještanje koda izvan petlje (Moving Invariants)

Ova tehnika se koristi kada se unutar petlje vrši proračun čiji je rezultat isti u svakoj iteraciji. Takve operacije nazivamo invarijantama.
Umjesto da procesor troši resurse na isto računanje hiljadu puta, mi taj proračun uradimo jednom prije ulaska u petlju.
Konkretno u metodi koju analiziram, unutar foreach petlje koja provjerava duplikate, nalazi se sljedeća linija:

```
if (f.getNazivFilma().ToLower() == naziv.ToLower())
```

Ono što je problematično ovdje jeste naziv.ToLower() funkcije. Varijabla naziv dolazi od korisničkog unosa i ne mijenja se unutar petlje. Ako u bazi postoji 1000 filmova, program će 1000 puta pretvarati isti naziv u mala slova, što bespotrebno alocira memoriju i troši procesorsko vrijeme.
Za postizanje boljih preformansi, potrebno je prosto za varijablu naziv premjestiti poziv ToLower() funkcije izvan tijela petlje, te ga dodijliti nekoj varijabli, koja ce se dalje koristiti za usporedbe unutar petlje.
Ovo radimo na sljedeći način:

```
string nazivLower = naziv.ToLower();

bool postoji = false;
foreach (var f in filmRepo.GetAll())
{
    if (f.getNazivFilma().ToLower() == nazivLower)
    {
        postoji = true;
        break;
    }
}
```

Provjera rezultata tuninga

Bilo je prvo potrebno dodati dosta filmova u bazu, jer je osnovna logika petlje da prolazi kroz postojece filmove i trazi duplikat. Dodavanje novog filma radjeno je 7000 puta.
Ispod se nalazi koristena testna metoda:

```
            var testFilmRepo = new FilmRepository();
        var testFilmService = new FilmService(testFilmRepo);

        for (int i = 0; i < 500; i++)
        {
            testFilmRepo.GetAll().Add(new Film("Postojeci Film " + i, "Akcija", 7.5, 2020));}

        Console.WriteLine("Pritisnite bilo koju tipku za početak mjerenja ...");
        Console.ReadKey();

        Stopwatch sw = new Stopwatch();
        sw.Start();

        for (int i = 0; i < 7000; i++)
        {
            testFilmService.dodajFilmTest("Novi Film " + i, "Drama", 8.8, 2023);}
        sw.Stop();

        Console.WriteLine($"Ukupno vrijeme: {sw.ElapsedMilliseconds} ms");
        Console.WriteLine("Mjerenje zavrseno");
```

Vrijeme izvršavanja prije i poslije tuninga petlje

```
C:\Users\Korisnik\OneDrive\D

Pritisnite bilo koju tipku za početak mjerenja (Baseline)...

--- REZULTATI MJERENJA ---
Ukupno vrijeme: 6225 ms
--------------------------

Mjerenje zavrseno
Pritisnite bilo koju tipku za ulazak u regularnu aplikaciju...
```

Slika 13: Vrijeme izvršavanja prije

```
Pritisnite bilo koju tipku za početak mjerenja (Baseline)...

--- REZULTATI MJERENJA ---
Ukupno vrijeme: 4160 ms
--------------------------

Mjerenje zavrseno
Pritisnite bilo koju tipku za ulazak u regularnu aplikaciju...
```

Slika 14: Vrijeme izvršavanja poslije

Alocirana memorija prije i poslije tuninga petlje

Program.cs | Snapshot #1 H....exe (14.32s) | FilmService.cs | FilmMate.csproj

Managed Memory — Compare With Baseline: None

Types | Insights

Filter types — Show Just My Code — Collapse small object types — Show dead objects — Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	191	25,832	26,192
RuntimeType	60	2,400	2,400
Func<Double>	27	1,728	1,728
PollingCounter	18	3,504	3,504
Func<Object, PropertyValue>	12	768	768
ScalarTypeInfo	11	1,112	1,152
WeakReference<EventProvider>	10	240	240
Total	**575**	**355,248**	

Slika 15: Alocirana memorija prije

Program.cs | Snapshot #1 H...e.exe (49.63s) | Snapshot #1 H....exe (32.57s) | FilmService.cs | FilmMate.csproj

Managed Memory

Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	191	25,832	26,192
RuntimeType	60	2,400	2,400
Func<Double>	27	1,728	1,728
PollingCounter	18	3,504	3,504
Func<Object, PropertyValue>	12	768	768
ScalarTypeInfo	11	1,112	1,152
WeakReference<EventProvider>	10	240	240
Total	**570**	**354,208**	

Slika 16: Alocirana memorija poslije

Nakon napravljene izmjene u metodi DodajFilm, ciklomatska kompleksnost ostala je ista(sto je, uoostalom, i očekivano, jer se broj grana nije promijenio). Uvedenom projmenom indeks održivosti smanjen je za 1.

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmMate (Debug)	81	232	2	35	1.057	379
Program	53	29	1	17	114	48
SortirajMeni	52	14	1	3	44	19
FilmMate.Data	88	22	1	11	98	32
FilmMate.Models	95	26	2	9	103	29
FilmMate.Services	73	141	1	24	698	251
FilmService	58	120	1	10	576	212
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	47	12		4	58	28

Slika 17: Metrike poslije Tuning petlje

2.4.3 Logičko pojednostavljivanje

U ovom dijelu cilj je objediniti višestruke if provjere (ocjena i godina) u jedan iskaz koristeći logičke operatore. Prvo, u dijelu koda gdje imamo da ugnježdena if-a:

```
    if (validOcjena)
            {
                if (ocjena < 1 || ocjena > 10)
                {
                    ocjena = 5;
                    Console.WriteLine("Neispravna ocjena, postavljena na 5.");
                }
            }
             else
            {
                ocjena = 5;
                Console.WriteLine("Neispravan format ocjene, postavljena na 5.");
            }
```

Ova dva ugnježdena iskaza provjere možemo vrlo jednostavno objedniti u sljedeći:

```
if (ocjena < 1 || ocjena > 10 || !validOcjena)
{
    ocjena = 5;}
```

Napravljena promjena koristi "short-circuit" logiku (——) koja omogućava procesoru da ranije prekine provjeru, štedeći CPU cikluse.
Izmjena je za svrhu testiranja urađena prvo na DodajFilmTest metodi, pa onda za procjenu MCcabe metrike i na analiziranu metodu.
Primjenom ovog tuninga, izvršena je ušteda vremena, što je prikzano na slici ispod. Obzirom da sama izmjena nema veze sa alokacijom memorijskih resursa, Memory usage je ostao isti kao na zadnjem snapshotu.

```
Pritisnite bilo koju tipku za početak mjerenja (Baseline)...

--- REZULTATI MJERENJA ---
Ukupno vrijeme: 2156 ms
--------------------------

Mjerenje zavrseno
Pritisnite bilo koju tipku za ulazak u regularnu aplikaciju...
```

Slika 18: Vrijeme izvršavanja nakon tuninga iskaza

Nakon urađene druge izmjene, indeks odrzivosti se povecao na 49, ciklomatska kompleksnost ostala je ista.

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmMate (Debug)	81	232	2	35	1.043	375
Program	53	29	1	17	114	48
SortirajMeni	52	14	1	3	44	19
FilmMate.Data	88	22	1	11	98	32
FilmMate.Models	95	26	2	9	103	29
FilmMate.Services	73	141	1	24	684	247
FilmService	59	120	1	10	562	208
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	49	12		4	50	25

Slika 19: Metrike nakon tuninga iskaza

2.4.4 Izbor boljeg algoritma ili struktura podataka

Trenutno, za svaki novi film, vi prolazimo kroz cijelu listu (foreach petlja). Ako ima n filmova, kompleksnost je $O(n)$.
Tuning: Ako u filmRepo iskoristimo HashSet¡string¿ za čuvanje naziva, provjera postojanja postaje trenutna, bez obzira da li ima 10 ili 10 miliona filmova.

Očekivani uticaj: Drastično smanjenje vremena izvršavanja kako baza raste.

U svrhu ovog, uveden je novi privatni atribut koji u konstruktoru popunimo svim postojecim filmovima:

```
private readonly HashSet<string> _naziviIndeks;

        public FilmService(IFilmRepository repo)
        {
            filmRepo = repo;

            _naziviIndeks = new HashSet<string>(
                filmRepo.GetAll().Select(f => f.getNazivFilma()),
                StringComparer.OrdinalIgnoreCase ); }
```

Onda je urađen tuning, umjeto prolaska kroz sve elemente, gledamo samo HashSet

```
if (_naziviIndeks.Contains(naziv))
            {
                Console.WriteLine("Film već postoji!");
                return;
            }
```

Potrebno je također nakon dodavanja filma update i hashset kako bi sadrzavao novi naziv.

Rezultati: Uvedenom promjenom napravljena je najveća ušteda vremena dosad, vrijeme izvršavanja testnog slučaja smanjeno je na 3ms!!
Iskorištenost memorije ostala je slična ako u prethodnim slučajevima, tj. ovdje nije psotignuta značajnija promjena:

```
Pritisnite bilo koju tipku za početak mjerenja (Baseline)...

--- REZULTATI MJERENJA ---
Ukupno vrijeme: 3 ms
--------------------------

Mjerenje zavrseno
Pritisnite bilo koju tipku za ulazak u regularnu aplikaciju...
```

Slika 20: Vrijeme izvršavanja nakon uvođenja HS

Managed Memory

Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	191	25,832	26,192
RuntimeType	60	2,400	2,400
Func<Double>	27	1,728	1,728
PollingCounter	18	3,504	3,504
Func<Object, PropertyValue>	12	768	768
ScalarTypeInfo	11	1,112	1,152
EventSource+OverrideEventProvider	10	640	2,000
Total	**571**	**354,080**	

Slika 21: Utrošak memorije nakon uvođenja HS

Zadnjom izmjenom također je smanjena ciklomatska kompleksnost za 1, i indeks odrzivosti je povećan na 50.

Code Metrics Results

Filter: None | Min: | Max:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmMate (Debug)	81	230	2	38	1.043	372
Program	53	29	1	17	114	48
SortirajMeni	52	14	1	3	44	19
FilmMate.Data	88	22	1	11	98	32
FilmMate.Models	95	26	2	9	103	29
FilmMate.Services	73	139	1	27	684	244
FilmService	59	118	1	13	562	205
filmRepo : IFilmRepository	100	0		1	1	0
_nazivIndeks : HashSet<string>	100	0		1	1	0
FilmService(IFilmRepository)	77	1		9	8	3
dodajFilm() : void	50	11		5	47	23

Slika 22: Ciklomatska kompleksnost nakon uvođenja HS

2.5 Refaktoring

U ovoj sekciji bit će urađen refaktoring s ciljem da povećamo indeks održivosti i smanjimo ciklomatsku kompleksnost metode. Tehnike refaktoringa koje će biti primijenjene na odabranu metodu, u skladu sa materijalima sa predavanja, su sljedeće:

- **Replace Magic Number with Symbolic Constant:** Fiksne vrijednosti minimalne i maksimalne ocjene (1 i 10), koje se pojavljuju u kodu, bit će zamijenjene imenovanim konstantama. Ovo olakšava održavanje koda ukoliko se poslovna pravila promijene.
- **Remove Control Flag:** Zastavica `bool postoji`, koja služi za prekidanje ili usmjeravanje toka, bit će uklonjena korištenjem direktnog povratka (`return`) ili ugrađenih funkcija nad kolekcijama, čime se pojednostavljuje kontrolni graf.
- **Rename Variable (Rename Method):** Varijable sa skraćenim imenima (npr. `kat` ili `g`) bit će preimenovane u deskriptivna imena (`kategorija`, `godinaIzdanja`) kako bi kod postao "samodokumentujući".
- **Extract Method (Izvlačenje metode):** Metoda `dodajFilm()` trenutno krši princip jedinstvene odgovornosti (SRP) jer vrši i unos, i validaciju, i provjeru duplikata. Logika za validaciju naziva i ocjene bit će izvučena u zasebne privatne metode kako bi osnovni tok bio čitljiviji.

Ovakav redoslijed refaktoringa je izabran iz vrlo jednostavnog razloga, tj. ideja je početi od "kozmetičkih" promjena koje čiste kod, a završavamo sa strukturnom promjenom za koju očekujemo da najviše utiče na metrike.

2.5.1 Checklist i očekivani rezultati refaktoringa

U sklopu refktoringa rađene su stavke iz tabele ispod, a u tabeli se nalazi i stvaka iz cehckliste za refaktoring kojoj pripadaju, kao i očekivani uticaji na metrike.

Promjena u kodu	**Stavka iz checkliste**	**Uticaj na Code Metrics**
Uvođenje konstanti	A primitive data type is overloaded	Povećan Maintainability Index
Preimenovanje `kat, g, ocj`	Attribute has a poor name	Poboljšana čitljivost i MI
Uklanjanje flaga `postoji`	A control structure is too deeply nested	Smanjen Cyclomatic Complexity
Izvlačenje validacije	A routine is too long	Drastičan pad CC po metodi

Tablica 15: Povezivanje refaktoringa sa checklistom i uticaj na metrike

2.5.2 Replace Magic Number with Symbolic Constant

U tvom kodu trenutno postoje provjere poput:

```
if(!vslidOcjena || ocjena < 1 || ocjena > 10)
```

Brojevi 1 i 10 su "magični" jer neko ko prvi put čita kod ne mora nužno znati šta oni predstavljaju. Promjena koju je potrebno izvršiti je: **Na nivou klase definisati konstante.**

```
private const double MinimalnaDozvoljenaOcjena = 1.0;
private const double MaksimalnaDozvoljenaOcjena = 10.0;
private const double PodrazumijevanaOcjena = 5.0;
```

Nakon ovog radimo izmjenu u kodu:

```
if (!validOcjena || ocjena < MinimalnaDozvoljenaOcjena || ocjena > MaksimalnaDozvoljenaOcjena)
```

Ono što se desilo uvođenjem ove izmjene jeste povećanje indeksa odrzivosti na 50.

2.5.3 Rename Variable (Preimenovanje)

Trenutno u kodu postoje skraćenice koje otežavaju čitanje. Obzirom da je pravilo, kao i dobra praksa, da ime treba da odražava namjenu, u ovoj sekciji bit će urađeno potrebno preimenovanje varijabli.
Radimo sljedeće tri izmjene:

kat → kategorijaFilma
godina → godinaIzdanja
ocjena → unesenaOcjena

Logika koda nije izmijenjena, a Maintability index se zaista povećao kao što je i očekivano.

2.5.4 Remove Control Flag

U analiziranoj metodi se trenutno koristi

$$\text{bool postoji} = \text{false};$$

a zatim prolazi kroz petlju da se postavi ta vrijednost.
To stvara dodatne grane u kontrolnom grafu.

Šta radimo:
Umjesto zastavice i if(postoji), koristimo direktan povratak. Isto je izbrisan if blok u kojem provjeravamo da li je vrijednost postoji postavljena na true

```
if (_naziviIndeks.Contains(naziv))
{
    Console.WriteLine("Film već postoji!");
    return; // Izlazimo odmah bez čuvanja stanja u flag
}
```

Ovom uvedenom izmjenom, radi brisanja nepotrebnog if bloka, prvi put je smanjena ciklomatska kompleksnost. Ona nakon urađene izmjene iznosi 10. Indeks održivosti se također povećao opet.

2.5.5 Extract Method

Ovo je posljednja, a ujedno i glavna promjena koja će biti urađena.
Na samom početku je spomenuto da metoda krši SRP (Single Responsibility Principle), s obzirom na to da ona istovremeno omogućava unos, validaciju i perzistenciju unesenih podataka.
Ono što ćemo uraditi u ovom dijelu jeste razbiti metodu na male dijelove. Svaka nova metoda će imati svoju (vrlo malu) ciklomatsku kompleksnost.

Izdvojeni dijelovi su sljedeći:

- **Metode za obradu ocjene:** Ove metode enkapsuliraju logiku provjere ranga i proces unosa.

```
private bool IsOcjenaValidna(double ocjena) {
    return ocjena >= MinimalnaDozvoljenaOcjena && ocjena <= MaksimalnaDozvoljenaOcjena;
}

private double DobaviValidnuOcjenu() {
    Console.Write("Ocjena (1-10): ");
    if (double.TryParse(Console.ReadLine(), out double unesenaVrijednost) &&
    ↪  IsOcjenaValidna(unesenaVrijednost)) {
        return unesenaVrijednost;
    }
    Console.WriteLine($"Neispravna ocjena, postavljena na {PodrazumijevanaOcjena}.");
    return PodrazumijevanaOcjena;
}
```

- **Metoda za provjeru postojanja filma:**

```
private bool FilmVecPostoji(string naziv) {
 Console.WriteLine("Film vec postoji!");
    return _naziviIndeks.Contains(naziv);
}
```

- **Metoda za unos i validaciju naziva:** Uključuje i provjeru minimalne dužine.

```
private string UnesiIValidirajNaziv() {
    Console.Write("Naziv: ");
    string naziv = Console.ReadLine() ?? "";
    if (string.IsNullOrWhiteSpace(naziv) || naziv.Length < 2) {
        Console.WriteLine("Naziv mora imati bar 2 karaktera!");
        return null;
    }
    return naziv;
}
```

- Metoda za spremanje filma u bazu

```
{
    filmRepo.GetAll().Add(new Film(naziv, kategorija, ocjena, godina));
    _naziviIndeks.Add(naziv);
    filmRepo.Sacuvaj();
    Console.WriteLine("Film uspješno dodat!");
}
```

Finalni izgled metode `dodajFilm()` nakon refaktoringa: Glavna metoda sada služi kao čisti "orkestrator" toka podataka.

```
    public void dodajFilm()
    {
        string naziv = UnesiIValidirajNaziv();
        if (naziv == null) return;

        if (FilmVecPostoji(naziv)) {

    return;
}
        string kategorijaFilma = Console.ReadLine();

        double konacnaOcjena = DobaviValidnuOcjenu();

        Console.Write("Godina: ");
        int godinaIzdanja = int.TryParse(Console.ReadLine(), out int gIzdanja) ? gIzdanja : 0;

        SpremiFilmUBazu(naziv, kategorijaFilma, konacnaOcjena, godinaIzdanja);
    }
```

2.5.6 Pregled postignutih rezultata

U ovoj sekciji osvrnut ćemo se na rezultate postignute refaktoringom. Na početku cjelokupnog testiranja, metoda je imala indeks održivosti 49 (čime se svrstavala u kategoriju metoda „teških za održavanje”) i ciklomatsku kompleksnost 12.
Najveća izmjena nabolje izvršena je procesom refaktoringa.
Najveći utvrđeni problem unutar metode, čije je rješavanje ključno utjecalo na poboljšanje metrika, predstavljalo je kršenje *Single Responsibility* principa.

Nakon izvršenog refaktoringa i optimizacije koda, ciklomatska kompleksnost metode smanjena je na 4, dok je indeks održivosti porastao na **67**.
Metode sa indeksom održivosti 67 pripadaju „zelenoj zoni”, koja označava kôd visoke čitljivosti i niske kognitivne složenosti. Za razliku od graničnih vrijednosti, ovaj rezultat pruža neophodnu sigurnosnu marginu, osiguravajući da kôd ostane održiv čak i nakon manjih budućih nadogradnji. Time je postignut primarni cilj procesa verifikacije: transformacija kompleksne i riskantne metode u stabilnu i modularnu komponentu sistema *FilmMate*.
Krajni rezultati vidljivi su an slici ispod:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmService	62	120	1	13	569	206
filmRepo : IFilmRepository	100	0		1	1	0
_nazivIndeks : HashSet<string>	100	0		1	1	0
FilmService(IFilmRepository)	77	1		9	8	3
MinimalnaDozvoljenaOcjena : double	93	0		0	1	1
MaksimalnaDozvoljenaOcjena : double	93	0		0	1	1
PodrazumijevanaOcjena : double	93	0		0	1	1
IsOcjenaValidna(double) : bool	90	2		0	4	1
FilmVecPostoji(string) : bool	93	1		1	4	1
UnesiIValidirajNaziv() : string	79	2		1	6	3
DobaviValidnuOcjenu() : double	70	3		1	13	6
SpremiFilmUBazu(string, string, double, int) : void	74	1		5	7	4
dodajFilm() : void	67	4		1	18	7

Slika 23: Krajnji rezultati

3 Metoda obrisiFilm() - Nedim Omanović

U ovoj sekciji bit će izvršena analiza metode obrisiFilm() iz FilmService klase.
Pomoću Code Metrics alata utvrđeno je da ova metoda ima ciklomatsku složenost koja zadovoljava uvjete postavljene u zadatku, pa je upravo iz toga razloga ona izabrana za analizu.

3.1 Opis metode

Odabrana metoda omogućava korisniku da obriše postojeći film iz repozitorija (filmRepo). Osim osnovne funkcionalnosti brisanja, metoda implementira višeslojnu validaciju i sigurnosne mehanizme za zaštitu podataka sa ocjenama.
Ispod se nalazi kod metode, kako bismo se mogli referencirati na neke njene dijelove kasnije pri analizi.

```
public void obrisiFilm()
{
    Console.Write("Unesite naziv filma: ");
    string naziv = Console.ReadLine() ?? "";
    if (string.IsNullOrWhiteSpace(naziv))
    {
        Console.WriteLine("Naziv filma ne može biti prazan.");
        return;
    }
    Film filmZaBrisanje = null;
    bool pronadjen = false;
    foreach (var f in filmRepo.GetAll())
    {
        if (f.getNazivFilma().ToLower() == naziv.ToLower())
        {
            filmZaBrisanje = f;
            pronadjen = false;
            break;
        }
    }
    if (pronadjen)
    {
        bool imaOcjene = filmZaBrisanje.getOcjene().Count > 0;
        string upozorenje = imaOcjene
            ? $"PAŽNJA: Film '{filmZaBrisanje.getNazivFilma()}' ima
            ↪ {filmZaBrisanje.getOcjene().Count} ocjena. "
            : "";
        Console.WriteLine($"Jeste li sigurni da želite obrisati {filmZaBrisanje.getNazivFilma()}?
        ↪ {upozorenje}(D/N)");
        string potvrda = Console.ReadLine()?.ToLower() ?? "";
        if (potvrda == "d")
        {
            if (imaOcjene)
            {
                Console.Write("Brisanjem se gube sve ocjene! Unesite 'OBRISI' za finalnu potvrdu:
                ↪ ");
                string finalnaPotvrda = Console.ReadLine() ?? "";
                if (finalnaPotvrda == "OBRISI")
                {
                    filmRepo.GetAll().Remove(filmZaBrisanje);
                    filmRepo.Sacuvaj();
                    Console.WriteLine("Film obrisan!");
                }
```

```
41                  else
42                  {
43                      Console.WriteLine("Brisanje otkazano zbog neuspjele finalne potvrde.");
44                  }
45              }
46              else
47              {
48                  filmRepo.GetAll().Remove(filmZaBrisanje);
49                  filmRepo.Sacuvaj();
50                  Console.WriteLine("Film obrisan!");
51              }
52          }
53          else
54          {
55              Console.WriteLine("Brisanje otkazano.");
56          }
57      }
58      else
59      {
60          Console.WriteLine("Film nije pronađen!");
61      }
62  }
```

3.2 McCabe metrika

3.2.1 Proračun McCabe metrike

Formula koja će biti primijenjena za računanje McCabe metrike je sljedeća:

$$V(G) = e - n + 2p$$

Značenje parametara je sljedeće:

1. **e** je broj ivica (tokova) u grafu programskog toka,
2. **n** je broj čvorova u grafu programskog toka,
3. **p** je broj formiranih neovisnih dijelova grafa.

Konkretno, u kontekstu metode koja se analizira, imamo da je:

1. **p=1** jer je u pitanju analiza jedne metode,
2. **n=17** čvorova u kontrolnom grafu,
3. **e=28** usmjerenih grana u kontrolnom grafu

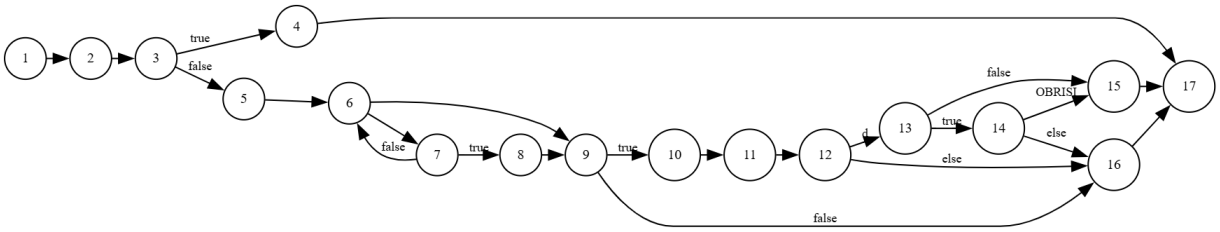
Uvrštavanjem u formulu dobijamo:

$$V(G) = 28 - 17 + 2 \times 1$$

$$V(G) = 13 \tag{3}$$

3.2.2 Kontrolni graf

U cilju validacije urađenog proračuna ciklomatske složenosti, na slici ispod je kontrolni graf. Na osnovu ovog grafa izvršena je identifikacija čvorova i ivica, što služi kao osnova za primjenu McCabeove formule.



Slika 24: Kontrolni dijagram metode obrisiFilm()

3.2.3 Poređenje rezultata

Nakon manuelnog proračuna, izvršena je analiza pomoću alata *Code Metrics* u Visual Studio. Rezultati su prikazani u nastavku:

Code Metrics Results

Filter: None Min: Max:

Hierarchy	Maintainability In...	Cyclomatic Com...	Depth of Inherita...	Class Coupling	Lines of Source c...	Lines of Executable c...
FilmService	59	110	1	10	523	194
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	46	13		4	70	30
azurirajFilm() : void	56	7		4	39	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4
OcijeniFilmGledaoca() : void	51	10		4	57	22

Slika 25: Code Metrics za metodu obrisiFilm()

Na osnovu slike, vidimo da se izračunata ciklomatska kompleksnost poklapa sa prethodno izračunatom.

Kvalitet koda: Vrijednost $V(G) = 13$ spada u opseg 10–20, što se prema McCabeovoj skali i Jonesovim istraživanjima smatra umjerenim rizikom i alarmom za pojačanu kontrolu. Budući da prelazi preporučenu granicu jednostavnosti (< 10), metoda se klasifikuje kao kompleksna. Osnovni razlog za ovakvu kompleksnost leži u višeslojnoj validaciji i sigurnosnim mehanizmima: metoda provjerava da li naziv postoji, zatim da li film postoji u bazi, provjerava da li film ima ocjene, zahtijeva potvrdu korisnika, te dodatnu potvrdu ako postoje ocjene. Takođe, metoda krši princip pojedinačne odgovornosti (SRP) jer vrši interakciju s konzolom, pretragu u repozitoriju, validaciju poslovnih pravila i perzistenciju.

Poteškoće u testiranju: Ciklomatska složenost direktno definiše broj nezavisnih putanja koje je potrebno testirati. Vrijednost $V(G) = 13$ znači da je potrebno pripremiti najmanje 13 jedinstvenih testnih slučajeva kako bi se postigla potpuna pokrivenost grana (branch coverage). Dodatnu poteškoću predstavlja direktna ovisnost o Console klasi, što onemogućava automatizovano jedinično testiranje bez manuelnog unosa ili kompleksnog preusmjeravanja standardnog ulaza/izlaza.

Indeks održivosti (MI = 46): Metoda se sa indeksom održivosti od 46 nalazi u "žutoj zoni" (ispod 65), što je klasifikuje kao metodu otežanu za održavanje. Ovako niska vrijednost je rezultat kombinacije visoke ciklomatske složenosti i broja linija koda (70 linija izvornog koda). Postojeća struktura sa višestrukim ugniježđenim if blokovima (dubina 4) otežava buduće izmjene u logici potvrde brisanja. Svaka promjena u procesu validacije ili sigurnosnim provjera zahtijeva modifikaciju ove metode, što povećava rizik od uvođenja regresijskih grešaka.

3.3 White-box testiranje

U ovoj sekciji bit će provedeno White-box testiranje, kako bi se ispitala interna struktura i logika odabrane metode, te otkrile postojeće greške. Budući da metoda ima ciklomatsku složenost 13, ona je idealan kandidat za White-box testiranje.
Obzirom na njenu strukturu, gotovo sve tehnike White-box testiranja obrađene na predavanjima mogu se primijeniti, uključujući obuhvat iskaza, obuhvat grana/odluka, obuhvat uslova, obuhvat petlji i obuhvat toka podataka.
Tehnika MCDC također je primjenjiva, ali je složenija i manje praktična za ovu metodu. Potpuni obuhvat svih mogućih puteva izvršavanja nije praktično izvediv zbog velikog broja kombinacija, stoga će se fokus testiranja biti stavljen na linearno nezavisne puteve.

3.3.1 Obuhvat iskaza/linija

Ova testna strategija ima težnju da dizajnira testne slučajeve tako da se svaki iskaz, tj. svaka linija koda izvrši barem jednom.
Cilj je da postignemo 100% pokrivenost iskaza uz *minimalan broj testnih slučajeva.*
Za metodu obrisiFilm potrebna su minimalno 6 testnih slučajeva kako bismo bili sigurni da se svaka linija izvršila barem jednom.

U tabeli ispod prikazani su ti testni scenariji:

	Opis scenarija	**Ulazni podaci (Konzola)**	**Očekivani ishod**
TC1	Prazan naziv	`naziv = ""`	Ispis greške i prekid (linije 5-8).
TC2	Film nije pronađen	`naziv = "NepostojeciFilm"` (film ne postoji u bazi)	Izvršavanje petlje bez pronalaska, ispis greške (linije 11-19, 57-60).
TC3	Brisanje filma bez ocjena sa potvrdom	`naziv = "Matrix"`, `potvrda = "d"` (film bez ocjena)	Uspješno brisanje (linije 20-52, grana else bez ocjena).
TC4	Brisanje filma sa ocjenama i uspješnom finalnom potvrdom	`naziv = "Inception"`, `potvrda = "d"`, `finalna = "OBRISI"`	Ulazak u if blok za ocjene, uspješno brisanje (linije 31-42).
TC5	Brisanje filma sa ocjenama ali neuspješnom finalnom potvrdom	`naziv = "Avatar"`, `potvrda = "d"`, `finalna = "CANCEL"`	Ulazak u else blok finalne potvrde (linije 43-46).
TC6	Korisnik odustaje od brisanja	`naziv = "Titanic"`, `potvrda = "n"`	Ulazak u else granu potvrde (linije 53-56).

Tablica 16: Testni slučajevi za obuhvat iskaza

3.3.2 Obuhvat odluka/grana

U ovoj sekciji fokus je na tačkama grananja u kodu. Cilj je osigurati da svaka odluka u programu barem jednom rezultuje vrijednošću True i barem jednom vrijednošću False.
U metodi obrisiFilm() imamo ključne tačke odluke, što možemo direktno očitati sa kontrolnog grafa iz prve sekcije.

1. **Null-coalescing operator (`??`) za naziv**: Provjera da li je `Console.ReadLine()` vratio `null`.
2. **Validacija naziva (`if`)**: Provjera `IsNullOrWhiteSpace` uslova.
3. **Kontrola petlje (`foreach`)**: Odluka o ulasku u tijelo petlje.
4. **Pronađen film (`if` uslov unutar petlje)**: Provjera da li se naziv unesenog filma podudara sa postojećim filmom u bazi.
5. **Provjera zastavice (`if (pronadjen)`)**: Odluka o nastavku procesa brisanja.
6. **Provjera ocjena (`bool imaOcjene`)**: Implicitna odluka za ternarni operator upozorenja.
7. **Ternarni operator za upozorenje**: Grananje za formiranje poruke upozorenja.
8. **Null-coalescing operator za potvrdu**: Sigurna obrada null vrijednosti.
9. **Provjera potvrde (`if (potvrda == "d")`)**: Odluka o nastavku ili otkazivanju brisanja.
10. **Provjera da li ima ocjena (`if (imaOcjene)`)**: Odluka o potrebi dodatne potvrde.
11. **Finalna potvrda (`if (finalnaPotvrda == "OBRISI")`)**: Konačna zaštita prije brisanja.

Kontrolni graf nam pomaže u identifikaciji svih putanja kroz metodu, tj. svih slučajeva koje moramo testirati.
Za potpuni obuhvat grana, moramo "proći" kroz svaku granu (strelicu) na grafu barem jednom.

Tablica 17: Minimalni set testova za 100% obuhvat grana

ID	Ulazni podaci i Preuslov	Putanja na grafu	Pokrivene odluke (Ishodi)
TC1	`naziv = null` ili `""`	1-2-3-4-17	**3(T - prazan naziv)**
TC2	`naziv = "Matrix"` (Baza: `[]`)	1-2-3-5-6-9-16-17	3(F), **7(F - petlja prazna), 9(F - pronadjen=false)**
TC3	`naziv = "Avatar"`, `potvrda = "n"` (Film postoji)	1-2-3-5-6-7-8-9-10-11-12-16-17	**7(T - pronađen), 9(T - pronadjen=true), 12(else - otkazano)**
TC4	`naziv = "Inception"`, `potvrda = "d"` (Film bez ocjena)	1-2-3-5-6-7-8-9-10-11-12-13-15-17	**12(d - potvrda), 13(F - nema ocjene)**
TC5	`naziv = "Titanic"`, `potvrda = "d"`, `finalna = "OBRISI"` (Film sa ocjenama)	1-2-3-5-6-7-8-9-10-11-12-13-14-15-17	**13(T - ima ocjene), 14(OBRISI)**
TC6	`naziv = "Joker"`, `potvrda = "d"`, `finalna = "xyz"` (Film sa ocjenama)	1-2-3-5-6-7-8-9-10-11-12-13-14-16-17	**14(else - otkazana finalna)**

3.3.3 Obuhvat uslova

Pokrivenost uslova (condition coverage) je tehnika testiranja koja procjenjuje svaki pojedinačni logički uslov unutar složenih izraza.

U metodi obrisiFilm() postoje složeni uslovi koji će biti testirani:

```
if (string.IsNullOrWhiteSpace(naziv))
```

```
string upozorenje = imaOcjene ? "PAŽNJA..." : "";
```

- **Uslov 1:**

  C_1: *string.IsNullOrWhiteSpace(naziv)*

- **Uslov 2 (Ternarni):**

  C_2: *imaOcjene (filmZaBrisanje.getOcjene().Count ¿ 0)*

- **Uslov 3:**

  C_3: *potvrda == "d"*

- **Uslov 4:**

  C_4: *finalnaPotvrda == "OBRISI"*

Tablica 18: Testni slučajevi za obuhvat uslova

ID	Ulazni podaci	C_1	C_2	C_3	C_4	Cilj testa
TC1	`naziv = ""`	**T**	-	-	-	Testira $C_1 = T$.
TC2	`naziv = "Matrix"`	**F**	-	-	-	Testira $C_1 = F$.
TC3	Film bez ocjena	-	**F**	-	-	Testira $C_2 = F$.
TC4	Film sa ocjenama	-	**T**	-	-	Testira $C_2 = T$.
TC5	`potvrda = "d"`	-	-	**T**	-	Testira $C_3 = T$.
TC6	`potvrda = "n"`	-	-	**F**	-	Testira $C_3 = F$.
TC7	`finalna = "OBRISI"`	-	-	-	**T**	Testira $C_4 = T$.
TC8	`finalna = "xyz"`	-	-	-	**F**	Testira $C_4 = F$.

3.3.4 Obuhvat petlji

Za metodu obrisiFilm(), jedina petlja je foreach koja prolazi kroz listu filmova kako bi pronašla film za brisanje. U analizi obuhvata petlji (*Loop coverage*), primjenjujemo sistematizovanu strategiju testiranja jednostavne petlje kroz ključne scenarije:

- **0 prolaza:** Preskakanje unutrašnjosti petlje u potpunosti.
- **1 prolaz:** Izvršavanje tijela petlje tačno jednom.
- **2 prolaza:** Izvršavanje tijela petlje dva puta kako bi se provjerila stabilnost iteracije.
- **m prolaza:** Slučajan broj prolaza kroz petlju, pri čemu je $m < n$.
- **Raniji izlaz:** Testiranje break mehanizma.

Tablica 19: Testni slučajevi za obuhvat petlji (Loop coverage)

ID	Scenario (n)	Putanja na grafu	Opis i očekivani ishod
TC1	0 prolaza (Prazna lista)	$3 \to 5$	Kolekcija je prazna. Petlja se preskače, varijabla `pronadjen` ostaje `false`.
TC2	1 prolaz (Bez podudaranja)	$3 \to 4 \to 3 \to 5$	Lista sadrži jedan film koji nije traženi. Program jednom izvršava provjeru.
TC3	2 prolaza (Bez podudaranja)	$3 \to 4 \to 3 \to 4 \to 3 \to 5$	Lista sadrži dva filma, nijedan nije traženi.
TC4	m prolaza ($m < n$)	$3 \to (\dots) \to 4 \to 3 \to 5$	Testiranje sa prosječnim brojem elemenata bez pronalaska.
TC5	Raniji izlaz (`break`)	$3 \to 4 \to 5$	Pronalazak filma u bilo kojoj iteraciji. Iz čvora 4 se putem `break` direktno ide u čvor 5.

3.3.5 Obuhvat toka podataka

U sekciji Obuhvat toka podataka (Data Flow Coverage), fokus se pomjera na životni ciklus varijabli unutar koda.
Treba za svaku varijablu biti pokrivena putanja od njene definicije (DEF) do mjesta upotrebe (USE).

Varijabla	**Definicija (linija)**	**Upotreba (linija)**	**Opis/Tip upotrebe**
`naziv`	4	5, 14, 26	Validacija praznog stringa, poređenje u petlji.
`filmZaBrisanje`	10	16, 23, 26, 36, 48	Inicijalizacija null, dodjela objekta, pristup metodama.
`pronadjen`	11	17, 21	Inicijalizacija i provjera zastavice.
`imaOcjene`	23	24, 31	Određivanje poruke i toka izvršavanja.
`upozorenje`	24	26	Ternarni operator i prikaz korisniku.
`potvrda`	27	28	Parsiranje i odluka o brisanju.
`finalnaPotvrda`	33	34	Dodatna sigurnosna provjera.

Tablica 20: Analiza definicija i upotreba (def-use) ključnih varijabli

Validacija putanja kroz testove:

Svi identifikovani DEF-USE parovi su pokriveni testnim scenarijima koji su prethodno definisani u sekciji *Obuhvat iskaza* (Tabela 16):

- Varijabla `naziv` prati putanju od unosa do provjere u petlji kroz scenarije praznog naziva (**TC1**), nepostojećeg filma (**TC2**) i uspješnog pronalaska (**TC3-TC6**).
- Zastavica `pronadjen` testirana je u oba stanja: *False* u slučaju nepostojećeg filma (**TC2**), *True* u svim ostalim scenarijima.
- Varijabla `filmZaBrisanje` prolazi kroz null inicijalizaciju i dodjelu objekta, te korištenje u različitim granama.
- Kontrolne varijable `imaOcjene`, `potvrda` i `finalnaPotvrda` testirane su kroz različite scenarije potvrde i otkazivanja brisanja.

3.3.6 Obuhvat puteva

Obuhvat puteva (Path Coverage) je najstroži kriterij White-box testiranja. Za metodu obrisiFilm, fokus je stavljen na nezavisne puteve kroz scenario nula ili jedne iteracije petlje.

Tablica 21: Identifikacija nezavisnih putanja kroz metodu obrisiFilm()

Putanja	**Čvorovi na grafu**
P1	$1 \to 2 \to 3 \to 4 \to 17$
P2	$1 \to 2 \to 3 \to 5 \to 6 \to 9 \to 16 \to 17$
P3	$1 \to 2 \to 3 \to 5 \to 6 \to 7 \to 8 \to 9 \to 10 \to 11 \to 12 \to 16 \to 17$
P4	$1 \to 2 \to 3 \to 5 \to 6 \to 7 \to 8 \to 9 \to 10 \to 11 \to 12 \to 13 \to 15 \to 17$
P5	$1 \to 2 \to 3 \to 5 \to 6 \to 7 \to 8 \to 9 \to 10 \to 11 \to 12 \to 13 \to 14 \to 15 \to 17$
P6	$1 \to 2 \to 3 \to 5 \to 6 \to 7 \to 8 \to 9 \to 10 \to 11 \to 12 \to 13 \to 14 \to 16 \to 17$

3.4 Code tuning

Code Tuning (podešavanje koda) je praksa promjene-modifikacije konkretnog koda na način da se poboljšaju njegove performanse.

3.4.1 Problem pri tuningu i rješenje

Osnovni problem na koji sam naišao radeći code tuning bio je što metoda obrisiFilm() očekuje unos iz konzole. Za potrebe testiranja performansi, gdje je trebalo testirati brisanje mnogo filmova, ovakav način je neprihvatljiv. Zato sam napravio pomoćnu metodu obrisiFilmTest, koja je u potpunosti identična mojoj analiziranoj obrisiFilm metodi, tj. zadržava svu logiku i kompleksnost, ali je izmijenjena na način da direktno prima potrebne parametre, umjesto da radi sa konzolom.

Sve izmjene pri testiranju i izračuni rađeni su na ovoj metodi. Kod se nalazi ispod:

```
public void obrisiFilmTest(string naziv, string potvrdaUnos, string finalnaPotvrda)
{
    if (string.IsNullOrWhiteSpace(naziv))
    {
        return;
    }

    Film filmZaBrisanje = null;
    bool pronadjen = false;

    foreach (var f in filmRepo.GetAll())
    {
        if (f.getNazivFilma().ToLower() == naziv.ToLower())
        {
            filmZaBrisanje = f;
            pronadjen = true;
            break;
        }
    }

    if (pronadjen)
    {
        bool imaOcjene = filmZaBrisanje.getOcjene().Count > 0;

        if (potvrdaUnos.ToLower() == "d")
        {
            if (imaOcjene)
            {
                if (finalnaPotvrda == "OBRISI")
                {
                    filmRepo.GetAll().Remove(filmZaBrisanje);
                    filmRepo.Sacuvaj();
                }
            }
            else
            {
                filmRepo.GetAll().Remove(filmZaBrisanje);
                filmRepo.Sacuvaj();
            }
        }
    }
}
```

3.4.2 Izmještanje koda izvan petlje (Moving Invariants)

Ova tehnika se koristi kada se unutar petlje vrši proračun čiji je rezultat isti u svakoj iteraciji.

Unutar foreach petlje koja pronalazi film za brisanje, nalazi se sljedeća linija:

```
if (f.getNazivFilma().ToLower() == naziv.ToLower())
```

Problem je što se `naziv.ToLower()` poziva u svakoj iteraciji petlje, iako je naziv konstantan. Ako u bazi postoji 1000 filmova, program će 1000 puta pretvarati isti naziv u mala slova.

Optimizacija:

```
string nazivLower = naziv.ToLower();

bool pronadjen = false;
foreach (var f in filmRepo.GetAll())
{
    if (f.getNazivFilma().ToLower() == nazivLower)
    {
        filmZaBrisanje = f;
        pronadjen = true;
        break;
    }
}
```

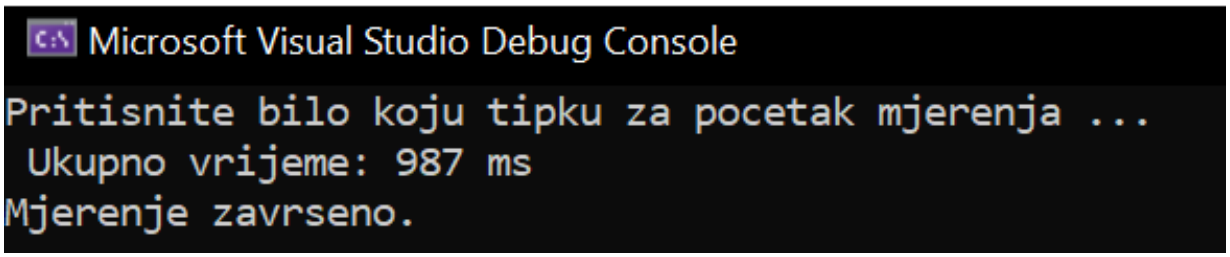
Provjera rezultata tuninga

Bilo je potrebno dodati filmove u bazu za testiranje. Testiranje je rađeno sa 500 postojećih filmova, a zatim je izvršeno brisanje 7000 puta.

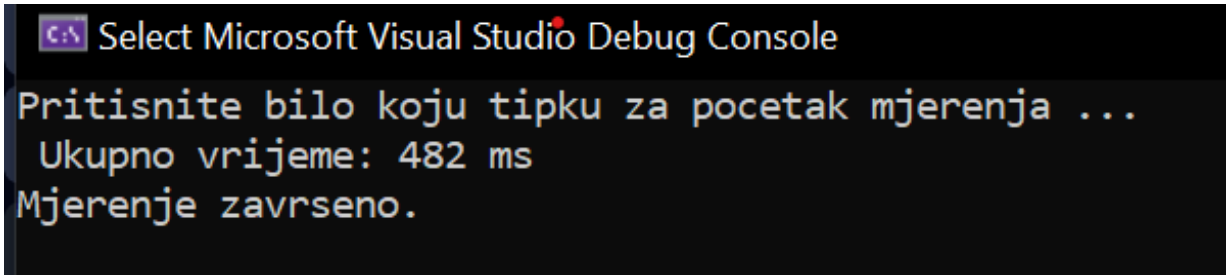
Ispod se nalazi korištena testna metoda:

```
var testFilmRepo = new FilmRepository();
var testFilmService = new FilmService(testFilmRepo);

// Dodaj 500 testnih filmova
for (int i = 0; i < 500; i++)
{
    testFilmRepo.GetAll().Add(new Film("Film " + i, "Drama", 7.5, 2020));
}

Console.WriteLine("Pritisnite bilo koju tipku za početak mjerenja ...");
Console.ReadKey();

Stopwatch sw = new Stopwatch();
sw.Start();

// Testiraj brisanje 7000 puta
for (int i = 0; i < 7000; i++)
{
    testFilmService.obrisiFilmTest("Film " + (i % 500), "d", "OBRISI");
}

sw.Stop();

Console.WriteLine($"Ukupno vrijeme: {sw.ElapsedMilliseconds} ms");
Console.WriteLine("Mjerenje završeno.");
```

Vrijeme izvršavanja prije i poslije tuninga petlje

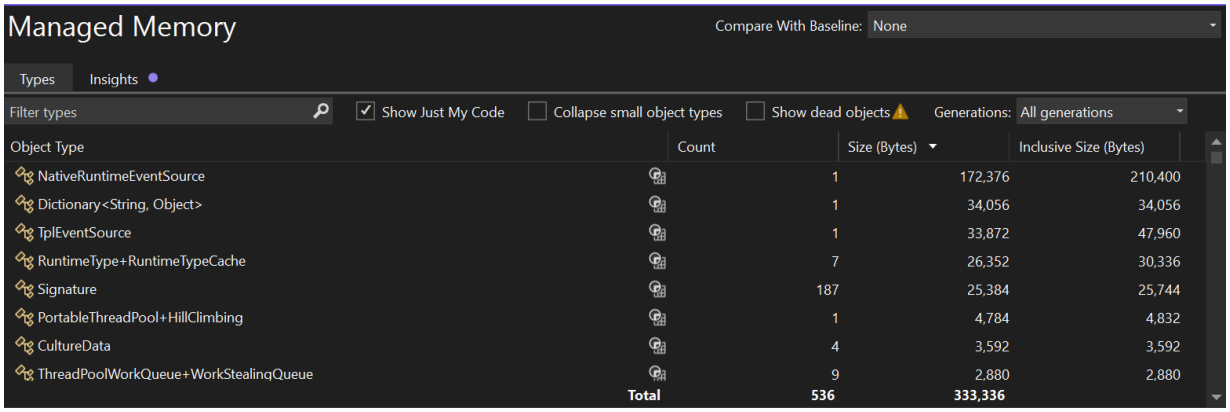


Slika 26: Vrijeme izvršavanja prije optimizacije petlje



Slika 27: Vrijeme izvršavanja poslije optimizacije petlje

Alocirana memorija prije i poslije tuninga petlje



Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
NativeRuntimeEventSource	1	172,376	210,400
Dictionary<String, Object>	1	34,056	34,056
TplEventSource	1	33,872	47,960
RuntimeType+RuntimeTypeCache	7	26,352	30,336
Signature	187	25,384	25,744
PortableThreadPool+HillClimbing	1	4,784	4,832
CultureData	4	3,592	3,592
ThreadPoolWorkQueue+WorkStealingQueue	9	2,880	2,880
Total	**536**	**333,336**	

Slika 28: Alocirana memorija prije optimizacije

Managed Memory

Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
NativeRuntimeEventSource	1	172,376	210,400
Dictionary<String, Object>	1	34,056	34,056
TplEventSource	1	33,872	47,936
RuntimeType+RuntimeTypeCache	7	26,352	30,312
Signature	187	25,384	25,744
PortableThreadPool+HillClimbing	1	4,784	4,832
CalendarData	1	2,744	2,744
Total	**517**	**329,736**	

Slika 29: Alocirana memorija poslije optimizacije

Nakon napravljene izmjene u metodi obrisiFilm, ciklomatska kompleksnost ostala je ista (što je očekivano, jer se broj grana nije promijenio). Indeks održivosti ostao je isti.

Code Metrics Results

Filter: None | Min: | Max:

Hierarchy	Maintainability I...	Cyclomatic Com...	Depth of Inherit...	Class Coupling	Lines of Source c...	Lines of Executable ...
FilmMate.Models	95	26	2	9	103	29
FilmMate.Services	73	139	1	24	683	251
FilmService	58	118	1	10	561	212
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	46	13		4	70	30

Slika 30: Metrike poslije optimizacije petlje

3.4.3 Logičko pojednostavljivanje

U ovom dijelu cilj je objediniti višestruke if provjere u jedan iskaz koristeći logičke operatore.

U dijelu koda imamo ugniježđene if iskaze:

```
if (potvrda == "d")
{
    if (imaOcjene)
    {
        if (finalnaPotvrda == "OBRISI")
        {
            // Brisanje
        }
    }
    else
    {
        // Brisanje
    }
}
```

Ova struktura se može pojednostaviti:

```
if (potvrda == "d" && (!imaOcjene || finalnaPotvrda == "OBRISI"))
{
    filmRepo.GetAll().Remove(filmZaBrisanje);
    filmRepo.Sacuvaj();
    Console.WriteLine("Film obrisan!");
}
```

Napravljena promjena koristi "short-circuit" logiku koja omogućava procesoru da ranije prekine provjeru.

```
Pritisnite bilo koju tipku za pocetak mjerenja ...
Ukupno vrijeme: 433 ms
Mjerenje zavrseno.
```

Slika 31: Vrijeme izvršavanja nakon logičkog pojednostavljivanja

Nakon urađene optimizacije, indeks održivosti se povećao za 4, ciklomatska kompleksnost je ostala ista.

Code Metrics Results

Filter: None Min: Max:

Hierarchy	Maintainability I...	Cyclomatic Com...	Depth of Inherit...	Class Coupling	Lines of Source c...	Lines of Executable
FilmService	59	118	1	10	538	203
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	50	13		4	46	22

Slika 32: Metrike nakon logičkog pojednostavljivanja

3.4.4 Izbor boljeg algoritma ili struktura podataka

Trenutno, za pronalazak filma, prolazimo kroz cijelu listu (foreach petlja). Ako ima n filmova, kompleksnost je $O(n)$.

Tuning: Ako u filmRepo iskoristimo HashSet¡string¿ ili Dictionary¡string, Film¿ za čuvanje filmova po nazivu, provjera postojanja postaje trenutna - $O(1)$.

U FilmService konstruktoru dodajemo:

```
private readonly Dictionary<string, Film> _filmIndeks;

public FilmService(IFilmRepository repo)
{
    filmRepo = repo;

    _filmIndeks = new Dictionary<string, Film>(
        filmRepo.GetAll().ToDictionary(
            f => f.getNazivFilma().ToLower(),
            f => f),
        StringComparer.OrdinalIgnoreCase
    );
}
```

Zatim mijenjamo logiku u obrisiFilm():

```
if (_filmIndeks.TryGetValue(naziv.ToLower(), out Film filmZaBrisanje))
{
    // Nastavi sa procesom brisanja
}
else
{
    Console.WriteLine("Film nije pronađen!");
    return;
}
```

Potrebno je također nakon brisanja filma update Dictionary:

```
filmRepo.GetAll().Remove(filmZaBrisanje);
_filmIndeks.Remove(naziv.ToLower());
filmRepo.Sacuvaj();
```

Rezultati:

```
Pritisnite bilo koju tipku za pocetak mjerenja ...
 Ukupno vrijeme: 37 ms
Mjerenje zavrseno.
```

Slika 33: Vrijeme izvršavanja nakon uvođenja Dictionary

Managed Memory

Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	187	25,384	25,744
RuntimeType	59	2,360	2,360
ScalarTypeInfo	10	1,008	1,048
Total	441	322,952	

Slika 34: Utrošak memorije nakon uvođenja Dictionary

Zadnjom izmjenom ciklomatska kompleksnost se smanjila za 2, indeks održivosti se povecao za 3.

Code Metrics Results

Filter: None | Min: | Max:

Hierarchy	Maintainability I...	Cyclomatic Com...	Depth of Inherit...	Class Coupling	Lines of Source c...	Lines of Executable ...
_filmIndeks : Dictionary<strir	100	0		2	1	0
FilmService(IFilmRepository)	73	1		10	10	4
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	49	11		5	64	25

Slika 35: Ciklomatska kompleksnost nakon uvođenja Dictionary

3.5 Refaktoring

U ovoj sekciji bit će urađen refaktoring s ciljem da povećamo indeks održivosti i smanjimo ciklomatsku kompleksnost metode. Tehnike refaktoringa koje će biti primijenjene su:

- **Replace Magic String with Symbolic Constant:** Stringovi poput "d" i "OBRISI" bit će zamijenjeni imenovanim konstantama.
- **Remove Control Flag:** Zastavica `bool pronadjen` bit će uklonjena korištenjem direktnog povratka ili Dictionary pristupa.
- **Rename Variable:** Varijable sa skraćenim imenima bit će preimenovane u deskriptivna imena.
- **Extract Method:** Metoda `obrisiFilm()` trenutno krši SRP jer vrši i unos, i pretragu, i validaciju, i brisanje. Logika bit će izvučena u zasebne privatne metode.

3.5.1 Checklist i očekivani rezultati refaktoringa

Promjena u kodu	Stavka iz checkliste	Uticaj na Code Metrics
Uvođenje konstanti	A magic string is used	Povećan Maintainability Index
Preimenovanje varijabli	Attribute has a poor name	Poboljšana čitljivost i MI
Uklanjanje flaga `pronadjen`	Control structure is too nested	Smanjen Cyclomatic Complexity
Izvlačenje metoda	A routine is too long	Drastičan pad CC po metodi

Tablica 22: Povezivanje refaktoringa sa checklistom

3.5.2 Replace Magic String with Symbolic Constant

U kodu trenutno postoje provjere poput:

```
if (potvrda == "d")
if (finalnaPotvrda == "OBRISI")
```

Stringovi "d" i "OBRISI" su "magični". Promjena:

```
private const string PotvrdaBrisanja = "d";
private const string FinalnaPotvrdaRijec = "OBRISI";
```

Nakon toga:

```
if (potvrda.ToLower() == PotvrdaBrisanja)
if (finalnaPotvrda == FinalnaPotvrdaRijec)
```

Ono što se desilo uvođenjem ove izmjene jeste povećanje indeksa odrzivosti na 49.

3.5.3 Rename Variable (Preimenovanje)

Trenutno se koriste sljedeće varijable koje treba preimenovati:

- `f` → `trenutniFilm`
- `potvrda` → `korisnickaPotvrdaBrisanja`
- `finalnaPotvrda` → `potvrdiZaSigurnostBrisanje`

Ono što se desilo uvođenjem ove izmjene jeste povećanje indeksa odrzivosti na 48.

3.5.4 Remove Control Flag

U metodi se trenutno koristi:

$$\text{bool pronadjen} = \text{false};$$

Ovo stvara dodatne grane u kontrolnom grafu.

Optimizacija sa Dictionary pristupom:

```
if (!_filmIndeks.TryGetValue(nazivLower, out Film filmZaBrisanje))
{
    Console.WriteLine("Film nije pronađen!");
    return;
}
// Nastavi sa brisanjem direktno, bez flaga
```

Ovom uvedenom izmjenom, prvi put je postignuto realno smanjenje ciklomatske kompleksnosti. Dok su prethodne optimizacije (uvođenje konstanti i preimenovanje varijabli) prvenstveno uticale na čitljivost i indeks održivosti, ova promjena direktno eliminiše suvišne kontrolne strukture iz metode.

Konkretno, uklonjena je kontrolna zastavica `bool pronadjen`, kao i cijela `foreach` petlja koja je iterirala kroz listu filmova. Umjesto toga, implementirana je direktna Dictionary pretraga koristeći metodu `TryGetValue()`, koja pruža konstantnu vremensku složenost *O(1)* nasuprot linearne složenosti *O(n)* kod iteracije kroz listu.

Ova izmjena eliminiše tri ključna čvora odluke iz kontrolnog grafa:

- Kontrolu foreach petlje (čvor 6-7)
- Uslovnu provjeru unutar petlje za podudaranje naziva (čvor 7)
- Finalnu provjeru zastavice `if (pronadjen)` (čvor 9)

Navedeni čvorovi su zamijenjeni sa jednim direktnim pozivom Dictionary metode, što rezultira smanjenjem ciklomatske kompleksnosti sa 13 na 11. Indeks održivosti se također povećao na 49, što predstavlja napredak ka prihvatljivijoj zoni održivosti.

3.5.5 Extract Method

Ovo je glavna promjena. Metoda krši SRP jer istovremeno radi unos, pretragu, validaciju i brisanje.

Izdvojeni dijelovi:

- **Metoda za unos i validaciju naziva:**

```
private string UnesiIValidirajNazivZaBrisanje()
{
    Console.Write("Unesite naziv filma: ");
    string naziv = Console.ReadLine() ?? "";
    if (string.IsNullOrWhiteSpace(naziv))
    {
        Console.WriteLine("Naziv filma ne može biti prazan.");
        return null;
    }
    return naziv;
}
```

- **Metoda za pronalazak filma:**

```
private Film PronadjiFilm(string naziv)
{
    if (_filmIndeks.TryGetValue(naziv.ToLower(), out Film film))
    {
        return film;
    }
    Console.WriteLine("Film nije pronađen!");
    return null;
}
```

- **Metoda za potvrdu brisanja:**

```
private bool DaLiKorisnikPotvrdjujeBrisanje(Film film)
{
    bool imaOcjene = film.getOcjene().Count > 0;
    string upozorenje = imaOcjene
        ? $"PAŽNJA: Film '{film.getNazivFilma()}' ima {film.getOcjene().Count} ocjena. "
        : "";

    Console.WriteLine($"Jeste li sigurni da želite obrisati {film.getNazivFilma()}?
    ↪  {upozorenje}(D/N)");
    string potvrda = Console.ReadLine()?.ToLower() ?? "";

    if (potvrda != PotvrdaBrisanja)
    {
        Console.WriteLine("Brisanje otkazano.");
        return false;
    }

    if (imaOcjene)
    {
        Console.Write("Brisanjem se gube sve ocjene! Unesite 'OBRISI' za finalnu potvrdu: ");
        string finalnaPotvrda = Console.ReadLine() ?? "";
        if (finalnaPotvrda != FinalnaPotvrdaRijec)
        {
            Console.WriteLine("Brisanje otkazano zbog neuspjele finalne potvrde.");
            return false;
        }
    }

    return true;
}
```

- **Metoda za izvršavanje brisanja:**

```
private void IzvrsiObrisFilma(Film film)
{
    filmRepo.GetAll().Remove(film);
    _filmIndeks.Remove(film.getNazivFilma().ToLower());
    filmRepo.Sacuvaj();
    Console.WriteLine("Film obrisan!");
}
```

Finalni izgled metode `obrisiFilm()` nakon refaktoringa:

```
public void obrisiFilm()
{
    string naziv = UnesiIValidirajNazivZaBrisanje();
    if (naziv == null) return;

    Film filmZaBrisanje = PronadjiFilm(naziv);
    if (filmZaBrisanje == null) return;

    if (DaLiKorisnikPotvrdjujeBrisanje(filmZaBrisanje))
    {
        IzvrsiObrisFilma(filmZaBrisanje);
    }
}
```

3.5.6 Pregled postignutih rezultata

U ovoj sekciji osvrnut ćemo se na rezultate postignute refaktoringom. Na početku cjelokupnog testiranja, metoda je imala indeks održivosti 46 (čime se svrstavala u kategoriju metoda „teških za održavanje") i ciklomatsku kompleksnost 13.

Najveća izmjena nabolje izvršena je procesom refaktoringa.
Najveći utvrđeni problem unutar metode, čije je rješavanje ključno utjecalo na poboljšanje metrika, predstavljalo je kršenje *Single Responsibility* principa. Metoda je istovremeno vršila interakciju sa konzolom, pretragu kroz repozitorij, višeslojnu validaciju i sigurnosne provjere, te konačno brisanje i perzistenciju podataka.

Nakon izvršenog refaktoringa i optimizacije koda, ciklomatska kompleksnost metode smanjena je sa 13 na **4**, što predstavlja dramatično smanjenje od 9 poena (69% redukcija). Istovremeno, indeks održivosti porastao je sa 46 na **70**, što predstavlja poboljšanje od 24.

Metoda sa indeksom održivosti 70 pripada „zelenoj zoni" (MI ¿ 65), koja označava kod visoke čitljivosti i niske kognitivne složenosti. Postignuta vrijednost od 70 ne samo da zadovoljava minimalne standarde održivosti, već pruža neophodnu sigurnosnu marginu, osiguravajući da kod ostane robustan čak i nakon budućih proširenja funkcionalnosti.

Dodatno, metoda je svedena sa originalnih 70 linija izvornog koda na samo 17 linija (smanjenje od 76%), sa svega 6 linija izvršnog koda. Dubina ugniježđavanja smanjena je sa 4 na 1, što dodatno olakšava razumijevanje kontrolnog toka.

Time je postignut primarni cilj procesa verifikacije: transformacija kompleksne i riskantne metode u stabilnu, modularnu i lako održivu komponentu sistema *FilmMate*. Glavna metoda `obrisiFilm()` sada služi kao čist „orkestrator" koji koordinira pozive specijaliziranih pomoćnih metoda, pri čemu svaka od njih ima jasnu pojedinačnu odgovornost.

Krajnji rezultati vidljivi su na slici ispod:

FilmMate | FilmMate.Services.FilmService | UnesiIValidirajNazivZaBrisanje()

```
87
88          filmRepo.GetAll().Add(new Film(naziv, kat, ocjena, godina));
89          filmRepo.Sacuvaj();
90          Console.WriteLine("Film uspješno dodat!");
91      }
92
93
        9 references
94      public void obrisiFilm()
95      {
96          string naziv = UnesiIValidirajNazivZaBrisanje();
97          if (naziv == null) return;
98
99          Film filmZaBrisanje = PronadjiFilm(naziv);
100         if (filmZaBrisanje == null) return;
101
102         if (DaLiKorisnikPotvrdjujeBrisanje(filmZaBrisanje))
103         {
104             IzvrsiObrisFilma(filmZaBrisanje);
105         }
106     }
107
        1 reference
108     private string UnesiIValidirajNazivZaBrisanje()
109     {
110         Console.Write("Unesite naziv filma: ");
```

70 % | 0 | 7 | Ln: 107 | Ch: 1 | SPC | CRLF

Code Metrics Results

Filter: None | Min: | Max:

Hierarchy	Maintainability I...	Cyclomatic Com...	Depth of Inherit...	Class Coupling	Lines of Source c...	Lines of Executable ...
PotvrdaBrisanja : string	93	0		0	1	1
FinalnaPotvrdaRijec : string	93	0		0	1	1
FilmService(IFilmRepository)	73	1		10	12	4
dodajFilm() : void	48	12		4	56	27
obrisiFilm() : void	70	4		1	17	6

Code Metrics Results | Developer PowerShell | Fine Code Coverage | Error List | Output

Slika 36: Krajnji rezultati nakon refaktoringa metode obrisiFilm()

4 Metoda OcijeniFilmGledaoca() - Irma Lemeš

U ovoj sekciji bit će izvršena analiza metode OcijeniFilmGledaoca() iz FilmService klase za koju je pomoću Code Metrics alata utvrđeno da ima prihvatljivu ciklomatsku složenost za analizu.

4.1 Opis metode

Metoda OcijeniFilmGledaoca() omogućava korisniku da ocijeni film koji se nalazi u repozitoriju (filmRepo). Na početku se provjerava da li postoje filmovi dostupni za ocjenjivanje, nakon čega korisnik unosi naziv filma uz osnovnu validaciju unosa. Metoda zatim pretražuje repozitorij kako bi pronašla odgovarajući film i prekida izvršavanje ukoliko film ne postoji. Nakon pronalaska filma, korisnik unosi numeričku ocjenu koja se validira po formatu i dozvoljenom rasponu vrijednosti. U slučaju ispravnog unosa, ocjena se dodaje filmu, podaci se trajno pohranjuju, te se korisniku prikazuje poruka o uspješnom izvršenju.

Ispod se nalazi kod metode:
baselinestretchbaselinestretch

```
        public void OcijeniFilmGledaoca()
{
    // KORIŠTENJE: filmRepo
    if (filmRepo.GetAll().Count == 0)
    {
        Console.WriteLine("Nema filmova za ocjenjivanje.");
        return;
    }

    Console.Write("Unesite naziv filma koji želite ocijeniti: ");
    string naziv = Console.ReadLine() ?? "";

    if (string.IsNullOrWhiteSpace(naziv))
    {
        Console.WriteLine("Naziv filma ne može biti prazan.");
        return;
    }

    Film filmZaOcjenjivanje = null;
    // KORIŠTENJE: filmRepo
    foreach (var f in filmRepo.GetAll())
    {
        if (f.getNazivFilma().ToLower() == naziv.ToLower())
        {
            filmZaOcjenjivanje = f;
            break;
        }
    }

    if (filmZaOcjenjivanje == null)
    {
        Console.WriteLine("Film nije pronađen!");
        return;
    }

    Console.Write($"Unesite ocjenu za '{filmZaOcjenjivanje.getNazivFilma()}' (1-10): ");
    int ocjena = 0;

    if (int.TryParse(Console.ReadLine(), out ocjena))
    {
        if (ocjena >= 1 && ocjena <= 10)
        {
            filmZaOcjenjivanje.DodajOcjenu(ocjena);
            // KORIŠTENJE: filmRepo
            filmRepo.Sacuvaj();
            Console.WriteLine($"Uspješno ste ocijenili film. Nova prosječna ocjena je
            ↪  {filmZaOcjenjivanje.getOcjena():F2}.");
        }
```

```
        else
        {
            Console.WriteLine("Ocjena mora biti u rasponu od 1 do 10.");
        }
    }
    else
    {
        Console.WriteLine("Neispravan unos za ocjenu.");
    }
}
```

4.2 McCabe metrika

4.2.1 Proračun McCabe metrike

Formula koja će biti primijenjena za računanje McCabe metrike je sljedeća:

$$V(G) = e - n + 2p$$

Značenje parametara je sljedeće:

1. **e** je broj ivica (tokova) u grafu programskog toka,
2. **n** je broj čvorova u grafu programskog toka,
3. **p** je broj neovisnih dijelova grafa.

Konkretno, u kontekstu metode `OcijeniFilmGledaoca()`, imamo da je:

1. $\mathbf{p = 1}$, jer se analizira jedna metoda,
2. $\mathbf{n = 18}$, što odgovara broju čvorova u kontrolnom grafu (ulaz, provjere uslova, petlja, validacije i izlazi),
3. $\mathbf{e = 26}$, što predstavlja ukupan broj usmjerenih grana između čvorova u kontrolnom grafu.

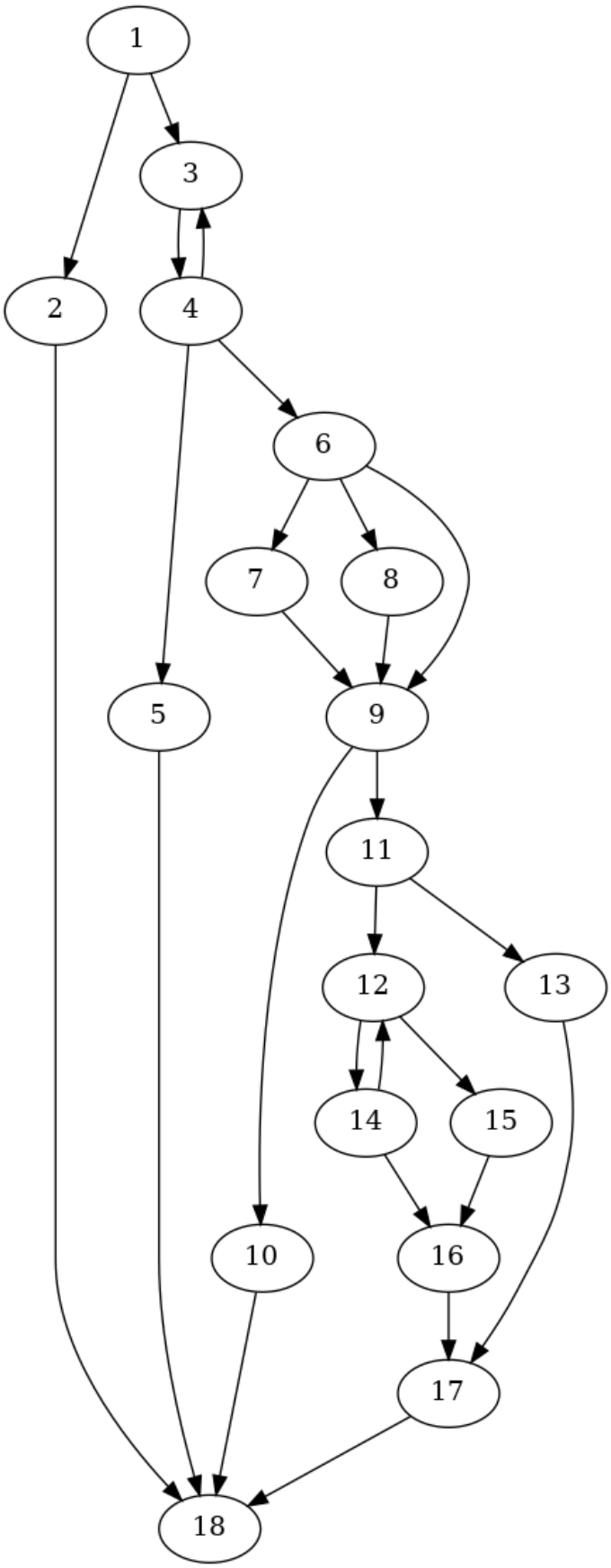
Uvrštavanjem vrijednosti u formulu dobijamo:

$$V(G) = 26 - 18 + 2 \cdot 1$$

$$V(G) = 10 \tag{4}$$

4.2.2 Kontrolni graf

U prilogu se nalazi kontrolni graf za navedenu metodu.



Slika 37: Kontrolni dijagram

4.2.3 Poređenje rezultata

Nakon ručnog proračuna, izvršena je analiza pomoću alata *Code Metrics* u Visual Studio. Rezultati su prikazani u nastavku:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	58	27
obrisiFilm() : void	46	13		4	74	30
azurirajFilm() : void	56	7		4	42	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4
OcijeniFilmGledaoca() : void	51	10		4	57	22
FiltrirajPretraziFilmove() : void	54	11		6	47	16
filtrirajPoNazivu() : List<Film>	66	4		7	16	7
filtrirajPoKategoriji() : List<Film>	66	4		7	16	7
filtrirajPoMinimalnojOcjeni() : List<Film>	63	4		7	22	9
Merge(List<Film>, List<Film>, Func<Film,	54	8		3	40	17
MergeSort(List<Film>, Func<Film, Film, bo	60	6		5	20	9
SortirajPoOcjeni(bool) : void	69	3		4	12	5
SortirajPoGodini(bool) : void	69	3		4	12	5

Slika 38: Code Metrics

Ručnim proračunom ciklomatske kompleksnosti metode OcijeniFilmGledaoca, pri čemu su složeni logički uslovi razloženi na elementarne odluke, odnosno gdje je operator tretiran kao dodatna odluka, dobijena je vrijednost V(G) = 10. Analizom iste metode pomoću alata Code Metrics također je utvrđena ciklomatska kompleksnost 10. Oba pristupa potvrđuju da metoda ima povećanu složenost i veći broj nezavisnih puteva izvršavanja. Dobijene metrike ukazuju na potrebu za refaktoringom kako bi se poboljšala održivost i čitljivost koda.

Kvalitet koda: Za vrijednost ciklomatske kompleksnosti V(G) = 10, možemo reći da spada u opseg 10–20, što se prema Jonesovim nalazima smatra alarmom za pojačanu kontrolu i potencijalnu podjelu metode. Ova metoda se ne smatra jednostavnom i lako razumljivom, već zahtijeva dodatnu pažnju tokom testiranja i održavanja.

Poteškoće u testiranju: Ciklomatska složenost direktno ukazuje na broj nezavisnih putanja kroz kod. Povećana složenost otežava testiranje, jer je za potpunu pokrivenost grana potrebno pripremiti veći broj testnih slučajeva. Dodatni problem predstavlja direktna zavisnost od Console klase, što otežava automatizovano jedinično testiranje.

Indeks održivosti (MI = 51) : Metoda se sa indeksom održivosti od 51 nalazi u "žutoj zoni" (ispod 65), što je klasifikuje kao metodu otežanu za održavanje. Malo viša složenost metode (48 linija koda) proizlazi iz miješanja par glavnih odgovornosti:

- Interakcija s korisnikom: Prikaz poruka i obrada korisničkog unosa putem više uslovnih provjera i rada s klasom Console.
- Poslovna logika: Pretraga repozitorija filmova, validacija ocjene i dodavanje nove ocjene filmu.
- Perzistencija podataka: Čuvanje izmjena u repozitoriju putem poziva metode Sacuvaj().

4.3 White box testiranje

4.3.1 Obuhvat iskaza/linija (Statement/line coverage)

Cilj obuhvata iskaza (statement/line coverage) je osigurati da se svaka izvršna linija koda u metodi `OcijeniFilmGledaoca()` izvrši barem jednom. Minimalan broj testnih slučajeva za potpuni obuhvat iskaza određen je analizom izvršnih linija koje se nalaze u međusobno isključivim granama izvršavanja. Zbog više return naredbi i alternativnih završetaka toka, potrebno je najmanje šest testnih slučajeva kako bi se svaka izvršna linija koda metode OcijeniFilmGledaoca() izvršila barem jednom. U nastavku je definisano 6 testnih slučajeva kroz tabelu.

	Opis scenarija	**Ulazni podaci (Konzola)**	**Očekivani ishod (obuhvaćene linije)**
TC1	Prazna lista filmova	Lista filmova je prazna	Izvršavanje provjere prazne liste i rani prekid metode (linije provjere liste i `return`).
TC2	Nevalidan naziv filma (prazan unos)	`naziv = ""`	Izvršavanje validacije naziva i rani prekid metode (linije provjere praznog unosa i `return`).
TC3	Film ne postoji u repozitoriju	`naziv = "NepostojeciFilm"`	Izvršavanje petlje bez pronalaska filma i ispis poruke o nepostojanju filma (linije `foreach` petlje i provjere `filmZaOcjenjivanje == null`).
TC4	Neispravan format ocjene	`naziv = "Titanic"`, `ocjena = "odličan"`	Ulazak u `else` blok nakon neuspjelog parsiranja ocjene (linije `TryParse` provjere i pripadajućeg `else` bloka).
TC5	Ocjena van dozvoljenog raspona	`naziv = "Titanic"`, `ocjena = "15"`	Izvršavanje provjere raspona ocjene i unutrašnjeg `else` bloka (linije provjere `ocjena >= 1 && ocjena <= 10`).
TC6	Validni podaci (Happy path)	`naziv = "Titanic"`, `ocjena = "9"`	Izvršavanje uspješnog toka metode: dodavanje ocjene, čuvanje u repozitoriju i ispis poruke o uspjehu (linije dodavanja ocjene i poziva `Sacuvaj()`).

Tablica 23: Testni slučajevi za obuhvat iskaza metode `OcijeniFilmGledaoca()`

4.3.2 Obuhvat odluka/grana (Decision/Branch Coverage)

U ovoj sekciji fokus je na tačkama grananja u kodu. Cilj obuhvata odluka (decision/branch coverage) jeste osigurati da svaka odluka u programu barem jednom rezultuje vrijednošću `True` i barem jednom vrijednošću `False`. Na taj način osigurava se da su sve grane izvršavanja metode testirane.

U metodi `OcijeniFilmGledaoca()` identifikovano je ukupno osam ključnih tačaka odluke, što je u skladu sa kontrolnim grafom predstavljenim u prethodnoj sekciji.

1. **Provjera postojanja filmova (`if`)**: Odluka da li je lista filmova prazna ili ne.
2. **Validacija naziva filma (`if`)**: Provjera da li je uneseni naziv prazan ili sadrži samo razmake.
3. **Kontrola petlje (`foreach`)**: Odluka o ulasku u tijelo petlje ili njenom završetku.
4. **Poklapanje naziva filma (`if` unutar petlje)**: Provjera da li se uneseni naziv podudara sa nazivom filma u repozitoriju.
5. **Provjera pronalaska filma (`if`)**: Odluka da li je film pronađen ili ne.
6. **Parsiranje ocjene (`int.TryParse`)**: Odluka da li je unesena ocjena ispravnog numeričkog formata.
7. **Provjera donje granice ocjene (`ocjena >= 1`)**: Dio složenog uslova za validaciju raspona.
8. **Provjera gornje granice ocjene (`ocjena <= 10`)**: Drugi dio složenog uslova za validaciju raspona.

Kontrolni graf omogućava identifikaciju svih mogućih putanja kroz metodu, odnosno svih ishoda navedenih odluka. Za potpuni obuhvat grana potrebno je da svaka od navedenih odluka barem jednom bude evaluirana na `True` i barem jednom na `False`.

Tablica 24: Minimalni set testnih slučajeva za potpuni obuhvat odluka

ID	Ulazni podaci i preuslov	Putanja na grafu	Pokrivene odluke (ishodi)
TC1	Lista filmova je prazna	1–2–18	1(T)
TC2	`naziv = ""` (lista nije prazna)	1–3–4–18	1(F), 2(T)
TC3	`naziv = "NepostojeciFilm"` (lista sadrži filmove)	1–3–4–6–9–11–18	2(F), 3(T), 4(F), 5(T)
TC4	`naziv = "Titanic"`, `ocjena = "abc"`	1–3–4–6–9–11–18	4(T), 5(F), 6(F)
TC5	`naziv = "Titanic"`, `ocjena = "15"`	1–3–4–6–9–11–12–15–16–17–18	6(T), 7(T), 8(F)
TC6	`naziv = "Titanic"`, `ocjena = "9"`	1–3–4–6–9–11–12–14–16–17–18	7(T), 8(T)

Iako pojedinačni testni slučajevi ne obuhvataju istovremeno oba ishoda svake odluke, kombinacijom svih testnih slučajeva osigurano je da svaka identifikovana odluka barem jednom bude evaluirana na vrijednost True i barem jednom na vrijednost False, čime je postignut potpuni obuhvat grana.

4.3.3 Obuhvat uslova (Condition Coverage)

Pokrivenost uslova (condition coverage) predstavlja tehniku white-box testiranja koja ispituje svaki pojedinačni logički uslov unutar složenih izraza u selekcionim iskazima i petljama. Cilj je osigurati da svaki elementarni logički uslov barem jednom poprimi vrijednost **True (T)** i barem jednom vrijednost **False (F)**.

U metodi `OcijeniFilmGledaoca()` postoji jedan složeni logički izraz koji sadrži više elementarnih uslova:

```
if (ocjena >= 1 && ocjena <= 10)
```

Kako bi se postigao potpuni obuhvat uslova, složeni izraz se razlaže na sljedeće elementarne logičke komponente:

- **Složeni uslov:**

  C_1: $ocjena >= 1$

  C_2: $ocjena <= 10$

U tabeli ispod nalaze se svi potrebni testni slučajevi za 100% obuhvat uslova:

Tablica 25: Testni slučajevi za obuhvat uslova metode `OcijeniFilmGledaoca()`

ID	**Ulazni podaci**	C_1	C_2	**Cilj testa**
TC1	`ocjena = 0`	**F**	**T**	Testira donju granicu uslova ($ocjena < 1$).
TC2	`ocjena = 15`	**T**	**F**	Testira gornju granicu uslova ($ocjena > 10$).
TC3	`ocjena = 5`	**T**	**T**	Testira validan raspon ocjene.

Na osnovu definisanih testnih slučajeva, svaki elementarni uslov unutar složenog izraza za validaciju raspona ocjene barem jednom poprima vrijednost **True** i barem jednom vrijednost **False**. Time je postignut potpuni obuhvat uslova (100% condition coverage) za analiziranu metodu.

4.3.4 Obuhvat petlji (Loop Coverage)

U metodi `OcijeniFilmGledaoca()` koristi se jedna petlja tipa `foreach`, koja prolazi kroz listu filmova u repozitoriju s ciljem pronalaska filma čiji naziv odgovara korisničkom unosu. Tokom iteracije, u slučaju pronalaska odgovarajućeg filma, petlja se prekida pomoću naredbe `break`.
Prilikom analize obuhvata petlji (*loop coverage*), primjenjuje se standardna strategija testiranja jednostavne petlje kroz sljedeće ključne scenarije:

- **0 prolaza:** Tijelo petlje se uopšte ne izvršava (lista je prazna).
- **1 prolaz:** Tijelo petlje se izvršava tačno jednom.
- **2 prolaza:** Tijelo petlje se izvršava dva puta.
- m **prolaza:** Petlja se izvršava proizvoljan broj puta, pri čemu je $m < n$.
- **Raniji izlaz:** Prekid izvršavanja petlje pomoću naredbe `break`.

U nastavku je prikazana tabela testnih slucajeva za obuhvat petlje:

Tablica 26: Testni slučajevi za obuhvat petlje u metodi `OcijeniFilmGledaoca()`

ID	Scenario (n)	Putanja na grafu	Opis i očekivani ishod
TC1	0 prolaza (prazna lista)	1–2–18	Lista filmova je prazna. `foreach` petlja se ne dostiže i metoda se završava ranijim izlazom.
TC2	1 prolaz (bez podudaranja)	1–3–4–6–9–10–18	Lista sadrži jedan film koji ne odgovara unesenom nazivu. Tijelo petlje se izvršava jednom bez `break`.
TC3	2 prolaza (bez podudaranja)	1–3–4–6–4–6–9–10–18	Lista sadrži dva filma, nijedan ne odgovara unesenom nazivu. Petlja se izvršava dva puta.
TC4	m prolaza ($m < n$)	1–3–4–(6–4)^m–6–9–10–18	Testiranje stabilnosti petlje za proizvoljan broj iteracija bez pronalaska filma.
TC5	Raniji izlaz (`break`)	1–3–4–6–9–10–18	Film se pronalazi u nekoj iteraciji, izvršava se `break` i petlja se prekida prije kraja liste.

Navedenim testnim slučajevima obuhvaćeni su svi karakteristični scenariji izvršavanja `foreach` petlje u metodi `OcijeniFilmGledaoca()`, uključujući preskakanje petlje, višestruke iteracije i raniji izlaz. Time je postignut potpuni obuhvat petlje (100% loop coverage).

4.3.5 Obuhvat toka podataka (Data Flow Coverage)

U sekciji obuhvata toka podataka (data flow coverage) fokus se pomjera sa samih putanja izvršavanja na životni ciklus varijabli unutar koda. Cilj ove tehnike je osigurati da je za svaku relevantnu varijablu pokrivena barem jedna putanja od njene definicije (DEF) do mjesta upotrebe (USE).
Ova vrsta testiranja posebno je pogodna za metode koje sadrže ugniježđene `if` iskaze i petlje, kao što je metoda `OcijeniFilmGledaoca()`, jer omogućava precizno praćenje kretanja podataka kroz različite grane izvršavanja i rano otkrivanje grešaka poput neinicijaliziranih ili pogrešno korištenih varijabli.

Varijabla	**Definicija (linija)**	**Upotreba (linija)**	**Opis / tip upotrebe**
`naziv`	9	10, 18	Validacija praznog unosa i poređenje naziva filma unutar `foreach` petlje.
`filmZaOcjenjivanje`	15	20, 24	Dodjela reference pronađenog filma i provjera da li je film pronađen.
`ocjena`	30	31, 33, 35	Parsiranje korisničkog unosa, validacija raspona i prosljeđivanje metodi za dodavanje ocjene.
`f`	16	18	Iteracijska varijabla `foreach` petlje, korištena za poređenje naziva filmova.

Tablica 27: Analiza definicija i upotreba (DEF–USE) varijabli u metodi `OcijeniFilmGledaoca()`

Validacija putanja kroz testne slučajeve:
Svi identifikovani DEF–USE parovi pokriveni su testnim scenarijima definisanim u prethodnim sekcijama (obuhvat iskaza, grana i uslova), čime je potvrđen ispravan tok podataka kroz sve relevantne grane izvršavanja metode:

- Varijabla `naziv` prati putanju od unosa sa konzole do validacije praznog unosa (**TC2**) i poređenja unutar petlje (**TC3**, **TC6**), čime su pokrivene sve njene upotrebe.
- Referenca `filmZaOcjenjivanje` testirana je u oba stanja: neinicijalizirana (**TC3** – film nije pronađen) i inicijalizirana (**TC6** – uspješno pronalaženje filma).
- Varijabla `ocjena` prolazi kroz putanje neuspješnog parsiranja (**TC4**), validacije raspona (**TC5**) i uspješnog dodavanja ocjene (**TC6**), čime su pokrivene sve njene upotrebe.
- Iteracijska varijabla `f` je testirana kroz različite scenarije prolaska kroz `foreach` petlju, uključujući višestruke iteracije i raniji izlaz pomoću `break` naredbe.

Na osnovu analize definicija i upotreba varijabli, kao i njihove validacije kroz prethodno definisane testne slučajeve, potvrđeno je da su sve relevantne DEF–USE putanje u metodi `OcijeniFilmGledaoca()` pokrivene. Time je ostvaren potpuni obuhvat toka podataka, čime se smanjuje rizik od grešaka povezanih sa nepravilnim rukovanjem podacima.

4.3.6 Obuhvat puteva (Path Coverage)

Obuhvat puteva (path coverage) predstavlja najstroži kriterij white-box testiranja, jer zahtijeva testiranje svih mogućih kombinacija grana od početka do kraja metode. Za razliku od obuhvata grana, gdje je dovoljno da se svaka odluka barem jednom evaluira na True i False, obuhvat puteva podrazumijeva prolazak kroz kompletne tokove izvršavanja.
U metodi `OcijeniFilmGledaoca()`, prisustvo `foreach` petlje uzrokuje potencijalno neograničen broj teorijskih puteva, zbog čega potpuni obuhvat svih mogućih putanja nije praktično izvodiv. Iz tog razloga, u okviru ovog zadatka fokus je stavljen isključivo na identifikaciju i testiranje **nezavisnih puteva**, u skladu sa McCabe metrikom.
Nezavisni putevi analizirani su uz pretpostavku nula ili jedne iteracije petlje, čime se dobijaju osnovni logički pravci izvršavanja metode.

Tablica 28: Identifikacija nezavisnih puteva kroz metodu `OcijeniFilmGledaoca()`

Putanja	**Čvorovi na kontrolnom grafu**
P1	$1 \to 2 \to 18$
P2	$1 \to 3 \to 4 \to 5 \to 18$
P3	$1 \to 3 \to 4 \to 6 \to 9 \to 10 \to 18$
P4	$1 \to 3 \to 4 \to 6 \to 9 \to 11 \to 13 \to 17 \to 18$
P5	$1 \to 3 \to 4 \to 6 \to 4 \to 6 \to 9 \to 11 \to 13 \to 17 \to 18$
P6	$1 \to 3 \to 4 \to 6 \to 9 \to 11 \to 12 \to 15 \to 16 \to 17 \to 18$
P7	$1 \to 3 \to 4 \to 6 \to 9 \to 11 \to 12 \to 14 \to 16 \to 17 \to 18$
P8	$1 \to 3 \to 4 \to 6 \to 9 \to 11 \to 12 \to 14 \to 12 \to 15 \to 16 \to 17 \to 18$
P9	$1 \to 3 \to 4 \to 6 \to 9 \to 11 \to 12 \to 14 \to 12 \to 14 \to 16 \to 17 \to 18$
P10	$1 \to 3 \to 4 \to 6 \to 9 \to 11 \to 12 \to 14 \to 16 \to 17 \to 18$

Identifikovani nezavisni putevi odgovaraju ciklomatskoj složenosti metode `OcijeniFilmGledaoca()`, gdje vrijednost V(G) = 10 označava minimalan broj nezavisnih putanja koje je potrebno testirati. Testiranjem navedenih puteva osigurava se potpuna pokrivenost osnovnih logičkih tokova izvršavanja metode.

4.4 Code Tuning

U ovoj sekciji izvršen je code tuning nad metodom `OcijeniFilmGledaoca()`. Cilj code tuning-a nije promjena funkcionalnosti metode, već optimizacija vremena izvršavanja i potrošnje memorije, uz zadržavanje iste logike i ciklomatske složenosti. Kako bi se omogućilo objektivno mjerenje performansi sa velikim brojem pokretanja, implementirana je test-verzija metode koja ne koristi konzolu, već prima potrebne parametre.

4.4.1 Test-verzija metode

U nastavku je prikazana metoda `OcijeniFilmGledaocaTest`, koja zadržava istu poslovnu logiku kao originalna metoda, ali omogućava automatizovano testiranje performansi.

```
public virtual bool OcijeniFilmGledaocaTest(
    List<Film> filmovi,
    string naziv,
    string ocjenaInput)
{
    if (filmovi.Count == 0)
        return false;

    if (string.IsNullOrWhiteSpace(naziv))
        return false;

    Film filmZaOcjenjivanje = null;

    foreach (var f in filmovi)
    {
        if (f.getNazivFilma()
              .Equals(naziv, StringComparison.OrdinalIgnoreCase))
        {
            filmZaOcjenjivanje = f;
            break;
        }
    }

    if (filmZaOcjenjivanje == null)
        return false;

    if (int.TryParse(ocjenaInput, out int ocjena))
    {
        if (ocjena >= 1 && ocjena <= 10)
        {
            filmZaOcjenjivanje.DodajOcjenu(ocjena);
            return true;
        }
    }

    return false;
}
```

4.4.2 Korištene tehnike

Nad metodom su primijenjene tri različite tehnike code tuning-a, u skladu sa kategorijama obrađenim na predavanjima. Fokus je bio na optimizaciji logičkih iskaza, izraza i algoritamske složenosti.

1. **Tehnika 1 – Tuning pristupa kolekcijama**
2. **Tehnika 2 – Tuning string izraza**
3. **Tehnika 3 – Tuning algoritma pretrage**

4.4.3 Tuning pristupa kolekcijama

Prije: Originalna metoda je višestruko pozivala metodu `filmRepo.GetAll()`, što može dovesti do nepotrebnog ponovnog učitavanja podataka, dodatnih alokacija memorije i povećanog vremena izvršavanja. **Poslije:** Lista filmova se dohvaća jednom i sprema u lokalnu varijablu, čime se eliminišu redundantni pozivi repozitorija. Ovom optimizacijom smanjen je broj poziva metoda i poboljšana je efikasnost rada sa kolekcijama, što se pozitivno odrazilo na vrijeme izvršavanja i potrošnju memorije.

```
var filmovi = filmRepo.GetAll();

foreach (var f in filmovi)
{
    if (f.getNazivFilma().Equals(
            naziv,
            StringComparison.OrdinalIgnoreCase))
    {
        filmZaOcjenjivanje = f;
        break;
    }
}
```

Vrijeme izvršavanja i alocirana memorija prije tuning pristupa kolekcijama

```
--- FilmMate Aplikacija Pokrenuta ---

--- BASELINE MJERENJE: OcijeniFilmGledaoca ---
Pritisnite bilo koju tipku za pocetak mjerenja...
Unesite naziv filma koji zelite ocijeniti: Matrix
Unesite ocjenu za 'Matrix' (1-10): 9
Uspjesno ste ocijenili film. Nova prosjecna ocjena je 8.50.

--- REZULTATI MJERENJA ---
Ukupno vrijeme: 7253 ms
------------------------

Pritisnite bilo koju tipku za izlaz...
```

Slika 39: Vrijeme izvršavanja prije

Managed Memory

Compare With Baseline: None

Types | Insights

Filter types | Show Just My Code | Collapse small object types | Show dead objects | Generations: All generations

Object Type	Count	Size (Bytes)	Inclusive Size (Bytes)
Signature	162	22,336	22,616
RuntimeType	48	1,920	1,920
ScalarTypeInfo	10	1,008	1,048
Func<Object, PropertyValue>	10	640	640
WeakReference<EventProvider>	8	192	192
EventSource+OverrideEventProvider	8	512	1,432
RuntimeType+RuntimeTypeCache	5	16,480	19,328
CultureInfo	4	512	2,264
WeakReference<EventSource>	4	96	96
Total	**345**	**300,616**	

Select a single type to view its reference graph.

Slika 40: Alocirana memorija prije

Vrijeme izvršavanja i alocirana memorija poslije tuning pristupa kolekcijama

```
Pritisnite bilo koju tipku za pocetak mjerenja...

Unesite naziv filma koji zelite ocijeniti: Matrix
Unesite ocjenu za 'Matrix' (1-10): 9
Uspjesno ste ocijenili film. Nova prosjecna ocjena je 8.50.

--- REZULTATI MJERENJA ---

---------------------------------------------
Ukupno operacija: 8000
VRIJEME:    4246 ms
MEMORIJA (razlika)):  293.6 KB
---------------------------------------------
```

Slika 41: Rezultati poslije

4.4.4 Tuning string izraza

Prije: Poređenje naziva filma vršeno je korištenjem metode `ToLower()`, što je u svakoj iteraciji petlje uzrokovalo kreiranje novih string objekata. **Poslije:** Poređenje je zamijenjeno metodom `Equals` uz korištenje `StringComparison.Ordina` čime se izbjegava nepotrebna alokacija memorije i smanjuje opterećenje garbage collectora. Ova tehnika je posebno efikasna u scenarijima sa velikim brojem elemenata u kolekciji.

```
if (f.getNazivFilma().Equals(
        naziv,
        StringComparison.OrdinalIgnoreCase))
{
    filmZaOcjenjivanje = f;
    break;
}
```

Vrijeme izvršavanja i alocirana memorija poslije tuning string izraza

```
--- BASELINE MJERENJE: OcijeniFilmGledaoca ---
Pritisnite bilo koju tipku za pocetak mjerenja...

Unesite naziv filma koji zelite ocijeniti: Matrix
Unesite ocjenu za 'Matrix' (1-10): 9
Uspjesno ste ocijenili film. Nova prosjecna ocjena je 8.50.

--- REZULTATI MJERENJA ---

--------------------------------------------
Ukupno operacija: 8000
VRIJEME:   1200 ms
MEMORIJA (razlika)):  175.8 KB
--------------------------------------------

Pritisnite bilo koju tipku za izlaz...
```

Slika 42: Rezultati poslije

4.4.5 Tuning algoritma pretrage

Prije: Pretraga filma u kolekciji vršena je linearnom pretragom sa vremenskom složenošću $O(n)$. **Poslije:** U testnoj verziji metode primijenjena je optimizacija zasnovana na indeksiranju kolekcije pomoću strukture `Dictionary`, čime se složenost pretrage smanjuje na $O(1)$. Ova optimizacija značajno poboljšava performanse metode u slučajevima kada se radi sa velikim brojem filmova.

```
var filmoviDict = filmovi.ToDictionary(
    f => f.getNazivFilma(),
    StringComparer.OrdinalIgnoreCase);

if (filmoviDict.TryGetValue(naziv, out Film filmZaOcjenjivanje))
{
    // Film pronađen
}
else
{
    return false;
}
```

Vrijeme izvršavanja i alocirana memorija poslije tuning algoritma pretrage

```
--- BASELINE MJERENJE: OcijeniFilmGledaoca ---
Pritisnite bilo koju tipku za pocetak mjerenja...

Unesite naziv filma koji zelite ocijeniti: Matrix
Unesite ocjenu za 'Matrix' (1-10): 9
Uspjesno ste ocijenili film. Nova prosjecna ocjena je 8.50.

--- REZULTATI MJERENJA ---

--------------------------------------------
Ukupno operacija: 8000
VRIJEME:   622 ms
MEMORIJA (razlika)):  161.4 KB
--------------------------------------------

Pritisnite bilo koju tipku za izlaz...
```

Slika 43: Rezultati poslije

4.4.6 Uticaj na metrike koda

Ciklomatska složenost (McCabe): Vrijednost ciklomatske složenosti poslije code tuning-a se poboljšala ($V(G) = 6$), pogotovo korištenjem pomoćne strukture `Dictionary`, što je očekivano. Značajno smanjenje ciklomatske složenosti bit će postignuto tek u fazi refaktoringa.

Indeks održavanja (Maintainability Index): Indeks održavanja (sada je 65) je nakon primjene code tuning-a također poboljšan, usljed smanjenja broja redundantnih izraza i jednostavnijih operacija nad podacima.

OcijeniFilmGledaoca(Dictionary<string, Fil	65	6	3
FiltrirajPretraziFilmove() : void	54	11	6
filtrirajPoNazivu() : List<Film>	66	4	7
filtrirajPoKategoriji() : List<Film>	66	4	7
filtrirajPoMinimalnojOcjeni() : List<Film>	63	4	7
Merge(List<Film>, List<Film>, Func<Film, I	54	8	3
MergeSort(List<Film>, Func<Film, Film, bo	60	6	5
SortirajPoOcjeni(bool) : void	69	3	4
SortirajPoGodini(bool) : void	69	3	4

Slika 44: Rezultati poslije

4.5 Refaktoring

U ovom dijelu bit će urađen refaktoring metode OcijeniFilmGledaoca() s ciljem da povećamo indeks održivosti i smanjimo ciklomatsku kompleksnost metode. Tehnike refaktoringa koje će biti primijenjene na odabranu metodu su sljedeće:

- **Replace Magic Number with Symbolic Constant:** Granice dozvoljene ocjene (1 i 10) zamijenjene su imenovanim konstantama, čime se olakšava održavanje i prilagodba poslovnih pravila.
- **Extract Method (Izvlačenje metode):** Logika za validaciju unosa, pronalazak filma i unos ocjene izdvojena je u zasebne metode, čime se metoda rasterećuje i povećava čitljivost.
- **Replace Conditional with Guard Clauses:** Složene `if/else` strukture zamijenjene su ranijim povratkom iz metode, čime se pojednostavljuje kontrolni tok.
- **Simplify Loop with Collection Operation:** Ručna iteracija kroz kolekciju zamijenjena je ugrađenim LINQ metodama, što smanjuje kompleksnost i rizik od grešaka.
- **Rename Variable / Method:** Imena varijabli i metoda prilagođena su da jasnije opisuju svoju namjenu.

Redoslijed refaktoringa započinje jednostavnijim, kozmetičkim izmjenama, dok se završava strukturnim promjenama koje imaju najveći uticaj na metrike kvaliteta koda.

4.5.1 Checklist i očekivani rezultati refaktoringa

U tabeli ispod prikazane su izvršene izmjene, stavke iz refaktoring checkliste kojima pripadaju, te očekivani uticaji na metrike.

Promjena u kodu	Stavka iz checkliste	Uticaj na Code Metrics
Uvođenje konstanti	Magic numbers are used	Povećan Maintainability Index
Izdvajanje validacije	A routine is too long	Smanjena CC po metodi
Guard clauses	Deeply nested conditionals	Smanjena Cyclomatic Complexity
LINQ pretraga kolekcije	Manual iteration over collection	Manji kontrolni graf
Preimenovanje	Poorly named variables	Poboljšana čitljivost

Tablica 29: Povezivanje refaktoringa sa checklistom i uticaj na metrike

4.5.2 Replace Magic Number with Symbolic Constant

U originalnoj metodi, granice dozvoljene ocjene bile su predstavljene magičnim brojevima:

```
if (ocjena >= 1 && ocjena <= 10)
```

Ovakav pristup otežava razumijevanje i održavanje koda. Rješenje je uvođenje imenovanih konstanti na nivou klase:

```
private const int MinimalnaOcjena = 1;
private const int MaksimalnaOcjena = 10;
```

Nakon toga, provjera postaje čitljivija i jasnija:

```
if (ocjena >= MinimalnaOcjena && ocjena <= MaksimalnaOcjena)
```

Ova izmjena rezultira povećanjem indeksa održivosti bez promjene funkcionalnosti.

4.5.3 Replace Conditional with Guard Clauses

Originalna metoda sadrži više ugniježđenih `if` struktura, što otežava praćenje toka izvršavanja. Primjenom guard clauses pristupa, provjere se izvršavaju odmah na početku metode ili bloka, uz rani izlazak u slučaju greške.

```
int ocjena = UnesiValidnuOcjenu();
if (ocjena == -1)
{
    return;
}

film.DodajOcjenu(ocjena);
filmRepo.Sacuvaj();
```

4.5.4 Extract Method

Najznačajnija promjena izvršena je izvlačenjem odgovornosti u zasebne metode. Time se metoda `OcijeniFilmGledaoca()` svodi na jasan tok.

Izdvojene metode su sljedeće:

- **Provjera postojanja filmova**

```
private bool ImaFilmova()
{
    if (filmRepo.GetAll().Count == 0)
    {
        Console.WriteLine("Nema filmova za ocjenjivanje.");
        return false;
    }
    return true;
}
```

- **Unos i validacija naziva filma**

```
private string UnesiNazivFilma()
{
    Console.Write("Unesite naziv filma koji želite ocijeniti: ");
    string naziv = Console.ReadLine() ?? "";

    if (string.IsNullOrWhiteSpace(naziv))
    {
        Console.WriteLine("Naziv filma ne može biti prazan.");
        return null;
    }
    return naziv;
}
```

- **Pronalaženje filma**

```
private Film PronadjiFilm(string naziv)
{
    return filmRepo.GetAll()
        .FirstOrDefault(f => f.getNazivFilma()
        .Equals(naziv, StringComparison.OrdinalIgnoreCase));
}
```

- **Unos i validacija ocjene**

```
private int UnesiValidnuOcjenu()
{
    Console.Write($"Unesite ocjenu ({MinimalnaOcjena}-{MaksimalnaOcjena}): ");
    if (int.TryParse(Console.ReadLine(), out int ocjena) &&
        ocjena >= MinimalnaOcjena && ocjena <= MaksimalnaOcjena)
    {
        return ocjena;
    }

    Console.WriteLine("Neispravan unos ocjene.");
    return -1;
}
```

Finalni izgled metode `OcijeniFilmGledaoca()` nakon refaktoringa:

```
public void OcijeniFilmGledaoca()
{
    if (!ImaFilmova()) return;

    string naziv = UnesiNazivFilma();
    if (naziv == null) return;

    Film film = PronadjiFilm(naziv);
    if (film == null)
    {
        Console.WriteLine("Film nije pronađen!");
        return;
    }

    int ocjena = UnesiValidnuOcjenu();
    if (ocjena == -1) return;

    film.DodajOcjenu(ocjena);
    filmRepo.Sacuvaj();

    Console.WriteLine(
        $"Uspješno ste ocijenili film. Nova prosječna ocjena je {film.getOcjena():F2}.");
}
```

4.5.5 Pregled postignutih rezultata

Prije refaktoringa, metoda je imala visoku ciklomatsku kompleksnost zbog velikog broja grananja i ugniježđenih kontrolnih struktura, kao i nizak indeks održivosti. Nakon izvršenog refaktoringa, ciklomatska kompleksnost prvobitne verzije metode smanjena je sa 10 na 5, dok je indeks održivosti porastao sa 51 na 61, što je dosta bolji rezultat. Iako se indeks održivosti i dalje nalazi u žutoj zoni, ostvareno poboljšanje predstavlja značajan korak ka povećanju čitljivosti i modularnosti koda. Dodatni rast indeksa bio bi moguć daljom dekompozicijom korisničke interakcije i potpunim odvajanjem poslovne logike od konzolnog interfejsa.

Uticaj na metrike koda

Hierarchy	Maintainability Index	Cyclomatic Comple...	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
FilmService	62	115	1	11	557	205
MinimalnaOcjena : int	93	0		0	1	1
MaksimalnaOcjena : int	93	0		0	1	1
filmRepo : IFilmRepository	100	0		1	1	0
FilmService(IFilmRepository)	96	1		1	5	1
dodajFilm() : void	48	12		4	58	27
obrisiFilm() : void	46	13		4	74	30
azurirajFilm() : void	56	7		4	42	16
izvrsiAzuriranje(Film) : void	67	13		3	38	4
UnesiNazivFilma() : string	71	3		1	14	6
ImaFilmova() : bool	75	2		4	9	4
PronadjiFilm(string) : Film	82	1		7	6	2
UnesiValidnuOcjenu() : int	65	4		1	18	8
OcijeniFilmGledaoca() : void	61	5		3	23	11
FiltrirajPretraziFilmove() : void	54	11		6	47	16
filtrirajPoNazivu() : List<Film>	66	4		7	16	7
filtrirajPoKategoriji() : List<Film>	66	4		7	16	7

Slika 45: Rezultati poslije Refactoringa

Primarni cilj refaktoringa je uspješno postignut: kompleksna i teško čitljiva metoda transformisana je u stabilnu i lako proširivu komponentu.